
Tecnologie Web

Appunti completi di teoria

Saverio Manco

Come usare questi appunti. Ogni capitolo corrisponde a un modulo del corso e ne copre *tutti* gli argomenti. I box colorati evidenziano: **definizioni** da saper riprodurre, **concetti da ricordare**, **errori e trappole**, **esempi** e, in **oro**, le **domande aperte già uscite** negli appelli passati, collocate accanto alla sezione che le copre.

Appunti creati a partire dalle slide ufficiali del corso

Indice

1	Uno sguardo d'insieme alle applicazioni web	6
1.1	Il World Wide Web e HTTP	6
1.2	Le URL	6
1.3	HTML e ipertesto (panoramica)	7
1.4	Risorse statiche e dinamiche	7
1.5	Anatomia di una transazione HTTP	8
1.6	Pagine interattive ed esecuzione lato client	8
1.7	Anatomia di un ciclo del browser	9
1.8	Il protocollo HTTP in dettaglio	9
1.8.1	La HTTP Request	9
1.8.2	La HTTP Response	10
1.9	Applicazioni web “full stack”	10
1.10	Front-end e back-end vs client-side e server-side	11
2	Il front-end: HTML	12
2.1	Il browser come ambiente di esecuzione	12
2.2	HTML: un linguaggio di annotazione	12
2.2.1	Struttura di un documento	13
2.2.2	Elementi a blocco e in linea	13
2.3	Collegamenti ipertestuali	13
2.4	Gli elementi principali	13
2.4.1	Testo	13
2.4.2	Tabelle	14
2.4.3	Contenuti multimediali	14
2.4.4	Elementi strutturali	14
2.5	I moduli (form) HTML	14
2.5.1	Il tag FORM	14
2.5.2	Gli elementi di input	15
3	Il front-end: CSS	16
3.1	Le regole di stile	16
3.1.1	Assegnare regole di stile a un HTML	16
3.1.2	Regole a cascata	16
3.2	Proprietà e valori	16
3.2.1	Tipi di valori	17
3.2.2	Proprietà composite	17
3.3	I selettori	17
3.3.1	Selettori semplici	17
3.3.2	Selettori composti	17
3.3.3	Selettori complessi	18
3.3.4	Liste di selettori	18
3.4	Il layout: display	18
3.4.1	display-inside: flow	19
3.4.2	display-inside: flex	19
3.4.3	display-inside: grid	19
3.4.4	display: none	20
3.5	Strumenti per CSS	20

4	JavaScript: le basi	21
4.1	Carta d'identità del linguaggio	21
4.1.1	JavaScript o ECMAScript?	21
4.1.2	Linguaggi interpretati, compilati, ibridi	21
4.2	Eseguire JavaScript nel browser	22
4.2.1	Collegare uno script a una pagina	22
4.3	Tipi, variabili e strutture di controllo	22
4.3.1	I tipi base	22
4.3.2	Variabili	22
4.3.3	Operatori e uguaglianza	23
4.3.4	Controllo di flusso	23
4.4	Array	23
4.5	Oggetti base	24
4.6	Classi (introduzione)	24
4.7	Funzioni	24
4.8	Scope e closure	25
4.9	Array e programmazione funzionale	26
4.10	Moduli: organizzare il codice su più file	27
4.10.1	Export	27
4.10.2	Import	27
5	JavaScript: prototipi, classi ed ereditarietà	28
5.1	I prototipi	28
5.1.1	Accedere e modificare i prototipi	28
5.1.2	Creare oggetti a partire da un prototipo	29
5.1.3	Determinare le proprietà di un oggetto	29
5.2	Metodi e binding di <code>this</code>	29
5.3	I costruttori: creare oggetti “con lo stampino”	30
5.4	Classi = costruttore + prototipo condiviso	30
5.4.1	Ereditarietà fra classi e catena dei prototipi	31
6	JavaScript e il DOM	32
6.1	Il Document Object Model	32
6.2	Interfacce e la DOM API	32
6.3	Navigare il DOM	33
6.3.1	Ricerca per tipo, classe, id	33
6.3.2	Attraversamento dell'albero	33
6.3.3	Ricerca per selettore	33
6.4	Modificare il DOM	34
6.4.1	Contenuti (modifica bulk)	34
6.4.2	Creazione, rimozione, inserimento	34
6.4.3	Classi e attributi	34
6.5	L'event loop asincrono single-thread	34
6.6	Gli eventi del DOM	35
6.6.1	Registrare un listener	35
6.7	Bubbling e capturing	35
6.7.1	Effetti di default	36
7	TypeScript e la gestione delle dipendenze	37
7.1	Carta d'identità di TypeScript	37
7.2	Package management: npm	37
7.2.1	Struttura di progetto npm	38

7.3	Configurare e compilare TypeScript	38
7.4	Dichiarazioni di tipo	39
7.5	Espressioni di tipo	39
7.5.1	Tipi base, array, tuple	39
7.5.2	Tipo unione	39
7.5.3	Tipo funzione	39
7.5.4	Le classi in TypeScript	40
7.5.5	Le interface	40
7.5.6	Dizionari indicizzati	41
7.5.7	Accessors: getter e setter	41
7.5.8	Generics	41
8	Richieste HTTP dal client: Fetch, Promise e JSON	43
8.1	La fetch API	43
8.1.1	AJAX e la storia della fetch	43
8.1.2	Perché fetch non prende una callback?	43
8.2	I problemi del meccanismo di callback	44
8.3	Le Promise	44
8.3.1	then, catch, finally	44
8.3.2	Combinare più operazioni asincrone	45
8.3.3	async / await	46
8.4	Il formato JSON	46
8.4.1	Serializzazione e deserializzazione	46
8.4.2	JSON nelle richieste e risposte HTTP	47
8.5	La server-side API	47
9	Applicazioni client-side con React	49
9.1	Architetture web: i flussi di informazione	49
9.2	React: caratteristiche principali	49
9.3	Componenti UI e gerarchie	50
9.3.1	Componenti dinamici: show/hide vs mount/unmount	50
9.3.2	Single Page Application	50
9.4	Il pattern MVVM	51
9.5	I componenti React: il ViewModel del MVVM	51
9.5.1	Espressioni JSX/TSX	51
9.6	Le props: l'input di un componente	52
9.7	Lo state: lo stato di un componente	53
9.7.1	Lo hook useState	53
9.7.2	I React Hooks	54
9.8	Delega di un evento al genitore	54
9.9	Form in React: binding bidirezionale	54
9.9.1	Tre tipologie di componenti form-like	55
9.10	Il ciclo di rendering di React	55
9.11	useEffect: gli effetti collaterali del rendering	55
9.11.1	useEffect + setState	56
9.11.2	La funzione di cleanup	56
10	Server HTTP e applicazioni server-side	57
10.1	Il Server HTTP	57
10.2	Gestione delle richieste: due modelli	57
10.3	HTTP Server vs Web Server	58
10.4	Le funzioni di un Server Web/HTTP	58

10.4.1	Fornire contenuti statici e dinamici	58
10.4.2	Virtual hosting	58
10.4.3	Autorizzazione e autenticazione	58
10.4.4	Caching	59
10.4.5	Bandwidth throttling	59
10.4.6	Rewrite engine	59
10.4.7	Logging	59
10.5	Server HTTP e applicazione server-side: chi contiene chi?	59
10.6	Caratteristiche generali di un'app server-side	60
10.7	Il ruolo dei framework server-side	60
11	Jakarta e le Servlet	62
11.1	Jakarta EE e il Web Profile	62
11.2	Architettura di esecuzione di un'app basata su Servlet	62
11.3	Cos'è una Servlet: la gerarchia di classi	63
11.4	Il ciclo di vita di una Servlet	63
12	Spring e Spring Boot	65
12.1	Il ruolo di Spring e di Spring Boot	65
12.1.1	App stand-alone e main class	65
12.1.2	Inizializzazione di progetto: SpringInitializr e Maven	66
12.1.3	Cenni al linguaggio XML	66
12.2	Le annotazioni Java	66
12.3	Il pattern MVC in Spring	67
12.4	Inversion of Control e Dependency Injection	67
12.5	L'architettura di Spring: IoC container e bean	67
12.6	Il @RestController: rispondere alle richieste HTTP	68
12.6.1	ResponseEntity<T>: inviare risposte JSON	69
12.6.2	Ricevere le proprietà della richiesta	69
12.7	I @Service: business logic	69
12.7.1	Suddivisione delle responsabilità	70
12.8	Entity e Data Transfer Object	70
13	Persistenza dei dati: JPA	71
13.1	Interfacciarsi a un DB relazionale	71
13.2	ORM, JPA, Hibernate	71
13.3	Le @Entity: descrivere entità e relazioni	71
13.3.1	Le annotazioni principali	72
13.3.2	Le relazioni	72
13.4	I @Repository e le operazioni CRUD	72
13.4.1	Le query derivate	72
13.5	Il ciclo di vita delle @Entity	73
13.5.1	@Transactional	73
13.5.2	save: persist o merge	73
14	CORS, sessioni, filtri e autenticazione	75
14.1	Cross-Origin Resource Sharing (CORS)	75
14.1.1	La preflight request	75
14.1.2	@CrossOrigin	75
14.2	Le sessioni	76
14.2.1	Sessioni tramite cookie	76
14.2.2	Gestione della sessione in Spring Boot	76
14.2.3	La collaborazione del client (e il CORS con credenziali)	77

14.3 I filtri	77
14.4 Basic Authentication con filtri e sessioni	78
15 Server-side app con Node.js ed Express	79
15.1 Node.js: il server HTTP di base	79
15.1.1 IncomingMessage: l'oggetto request	79
15.1.2 ServerResponse: l'oggetto response	79
15.2 Express	80
15.2.1 Il middleware in Express	80
15.2.2 I request handler in Express	80

1. Uno sguardo d'insieme alle applicazioni web

1.1 Il World Wide Web e HTTP

Definizione – World Wide Web

Il **World Wide Web** (WWW o “Web”) è un *servizio della rete Internet*: di tipo **client/server**, basato sul protocollo di livello applicativo **HTTP**, orientato alla pubblicazione e condivisione di contenuti e risorse digitali. Lo schema di base è: il **Web Client** (tipicamente un browser) invia una *richiesta HTTP* al **Web Server**, che risponde con una *risposta HTTP*.

Il Web nasce per condividere documenti e media: la richiesta HTTP specifica *quale risorsa* si vuole da *quale server*; la risposta “contiene” la risorsa, corredata di meta-informazioni che permettono al client di riprodurla al meglio.

Due tipologie di richieste (in prima battuta):

- **GET**: ottenere una risorsa. Deve indicare univocamente la risorsa, il server e l'origine (mittente), così che il server sappia a chi rispondere.
- **POST**: inviare dati al server perché li elabori o conservi. La destinazione è comunque una “risorsa”, ma deve essere una risorsa in grado di effettuare una computazione (uno script o applicativo eseguibile dal server, fatto per ricevere quei dati).

La **risposta HTTP** deve prima di tutto dire se la richiesta è andata a buon fine; in caso positivo contiene i dati (la risorsa per una GET, il risultato dell'elaborazione per una POST) e ne indica il *tipo*, perché il client possa renderli fruibili.

1.2 Le URL

Definizione – URL

Una **URL** (Uniform Resource Locator) è una stringa di testo con formato ben definito, in grado di identificare *univocamente* ogni risorsa disponibile sul Web:

`protocol://hostname[:port]/path[?querystring] [#fragment]`

Esempio: `http://www.organizzazione.it:80/utenti/profilo?name=picardi#phone`

- **Protocol**: il protocollo di livello applicativo (HTTP per il Web, ma le URL sono usate anche da FTP, SMTP, ecc.).
- **Domain/Host**: identifica lo specifico Web Server. Tipicamente tre parti: dominio di *terzo* livello o sottodominio (**www**, associato a uno specifico host), di *secondo* livello (**organizzazione**, associato alla rete di un'organizzazione) e di *primo* livello (**it**, area geografica e/o tipologia di organizzazione). “Dominio” generico = secondo + primo livello (**organizzazione.it**); i domini si acquistano da provider e si registrano presso il Registro del dominio di primo livello, a garanzia dell'univocità.
- **Port**: su uno stesso host possono rispondere più servizi, ognuno su una “porta” identificata da un numero. Non si specifica se è quella di default: **80** per HTTP, **443** per HTTPS (versione *sicura*, cifrata, di HTTP).

- **Path:** percorso per reperire la risorsa; segmenti separati da /. Può essere un percorso reale di cartelle/file oppure un *percorso logico* che identifica una funzione di un applicativo.
- **Query string:** testo opzionale che aggiunge informazioni come coppie chiave/valore; inizia con ?, coppie *c=v* separate da &. Es.: ?min=5&max=100&sort=alphabetical.
- **Fragment:** se la risposta è una pagina HTML, indica una collocazione precisa (*location*, definita nella pagina con un nome) a cui il browser deve posizionarsi; preceduta da #.

Definizione – DNS

Il **DNS** (Domain Name System) è il servizio di livello applicativo responsabile di tradurre lo *hostname* nel corrispondente *indirizzo IP*. HTTP usa i protocolli sottostanti TCP (trasporto) e IP (rete), che identificano gli host tramite indirizzi numerici: prima di inviare una richiesta HTTP, il client chiede al DNS server l'IP corrispondente allo hostname; solo a quel punto la richiesta può partire.

1.3 HTML e ipertesto (panoramica)

Prima del Web, Internet serviva già a scambiare risorse, ma in modo scomodo: bisognava conoscere l'host, collegarsi e scorrerlo cartella per cartella, con criteri di organizzazione diversi da host a host. Il Web nasce a fine anni '80 da due intuizioni:

1. le **URL**: un indirizzo univoco e uniforme per qualsiasi risorsa, indipendente dalla collocazione;
2. documenti **ipertestuali e multimediali**: un *ipertesto* è un testo multidimensionale, senza un'unica sequenza di lettura obbligatoria, con salti e rimandi fra cui il fruitore si muove liberamente; la multimedialità integra immagini, audio e video nel documento.

Definizione – HTML

HTML (HyperText Markup Language) è un linguaggio informatico ma *non* un linguaggio di programmazione: non istruisce un agente di calcolo su come elaborare dati. È un linguaggio di **markup** (annotazione): le pagine HTML sono documenti plain text contenenti *tag* (etichette) che indicano al browser come strutturare la pagina; ulteriori tag specificano link ipertestuali attivabili cliccando su porzioni di testo o aree (*hot spot*). È complementato da **CSS** (Cascading Style Sheet), con cui si comunica al browser l'aspetto grafico di ciascun elemento.

Le pagine HTML sono il “mattoncino fondamentale” del Web: contengono testo e media, e collegamenti ad altre risorse (media usati per visualizzare la pagina stessa, o rimandi ipertestuali ad altre pagine). È grazie a questi collegamenti che il Web è *navigabile* – l'immagine delle risorse sparse per il mondo e collegate fra loro richiama una ragnatela, da cui il nome “Web”.

1.4 Risorse statiche e dinamiche

Definizione – Risorsa statica vs dinamica

Una risorsa (o pagina) individuata da una URL è **statica** se corrisponde a un file già esistente, che il server deve semplicemente restituire al client (la URL rimanda a una risorsa fisica, anche se il path logico può non coincidere con il path nel file system). È **dinamica** se viene *calcolata sul momento*: la URL rimanda a un programma o script eseguibile, che effettua un calcolo e restituisce il risultato (sulla base dei dati nella richiesta HTTP, del file system, di basi di dati o di servizi di terze parti).

Esecuzione lato server. Un Web Server è un software installato sulla macchina host. Diverse tipologie di Web Server sanno eseguire programmi in diversi linguaggi (tipicamente interpretati

come JavaScript o ibridi come Java), contenendo al proprio interno un *agente di esecuzione* per quel linguaggio (es. una Java Virtual Machine). Si possono così scrivere applicativi **server-side**, che rispondono a determinate richieste HTTP su determinate URL attivando opportune funzioni di calcolo: la funzione attivata dipende dal *path*, dall'eventuale *query string* e dai *dati* della richiesta (se POST) – elementi che fanno anche da input del calcolo. L'output viene incapsulato nella risposta HTTP. Linguaggi diffusi lato server: JavaScript, Java, C#, PHP, Python, Ruby.

1.5 Anatomia di una transazione HTTP

Da ricordare – I 6 passi di una transazione HTTP

1. **Il client “riceve” una URL:** dall'utente in modo diretto (barra degli indirizzi) o indiretto (click su un link); come effetto di una precedente richiesta (una pagina HTML può richiedere altre N risorse, es. immagini); oppure generata dall'esecuzione di codice client-side.
2. **Il client chiede al DNS** l'indirizzo IP dell'host della URL.
3. **Il client invia la richiesta HTTP** a quell'IP; la richiesta contiene comunque la URL originale.
4. **Smistamento per porta:** l'host verifica che la porta specificata corrisponda effettivamente a un web server e non ad altro servizio.
5. **Il Web Server esamina la URL** (in particolare il path): (a) risorsa *statica* → la reperisce nel file system e la incapsula nella risposta; (b) risorsa *dinamica* → esegue il programma corrispondente e ne incapsula l'output (lo scopo può anche essere solo salvare un dato o notificare terzi: in ogni caso l'esito va comunicato nella risposta); (c) risorsa *inesistente* → risposta con codice d'errore (il celebre 404 Not Found).
6. **La risposta torna al client**, che la “spacchetta” determinando il tipo di dato: se è HTML la visualizza; immagini, audio, video, PDF sono gestiti autonomamente; tipi non gestiti vengono proposti in download.

1.6 Pagine interattive ed esecuzione lato client

Una pagina HTML è **passiva** quando può solo essere letta (al più si clicca un link per *navigare* altrove). È **interattiva** quando il browser “fa qualcosa” (oltre a navigare) in risposta alle azioni dell'utente – concretamente: esegue codice. Una concezione corretta di interattività prevede che l'aspetto della pagina si modifichi per dare conto che “qualcosa è stato fatto”. Si distingue: *interattività semplice* (effetto solo sulla visualizzazione, per migliorare la fruizione) e *interattività complessa* (vera elaborazione dati, con eventuali richieste verso un Web Server → si parla di **applicazione client-side**).

Esecuzione lato client. Ogni browser contiene un *motore di esecuzione* per JavaScript (linguaggio interpretato). Quando arriva una pagina con script, il browser lo esegue: l'effetto principale è “popolare” l'ambiente di esecuzione di *funzioni da eseguire in risposta a eventi* (click, tasti, movimenti del mouse, ecc.). L'ambiente mette a disposizione una rappresentazione a oggetti della pagina (il DOM), che il codice può modificare cambiando l'aspetto della pagina; può anche contenere oggetti e variabili che mantengono uno *stato*: così la pagina diventa la user interface di un vero applicativo **client-side**.

Da ricordare

Le richieste HTTP formulate da codice JavaScript hanno una peculiarità: i dati della risposta *non* vengono gestiti dal browser, ma **ricevuti dallo stesso codice che ha formulato la**

richiesta, che può elaborarli e usarli per modificare ciò che l'utente vede. Inoltre: il browser esegue *solo* JavaScript; le applicazioni client-side o sono scritte in JavaScript o devono essere *tradotte* in JavaScript prima dell'esecuzione.

1.7 Anatomia di un ciclo del browser

1. Il browser **ottiene una pagina HTML** (esito di una precedente transazione HTTP).
2. Determina se la pagina specifica **ulteriori risorse** necessarie alla *renderizzazione*: quasi sempre uno o più fogli di stile CSS (regole che istruiscono il browser sull'aspetto grafico), immagini o altri media e, se la pagina è interattiva, uno o più script JavaScript. Il browser le richiede con altrettante transazioni HTTP. Con HTML e CSS il browser **inizializza il DOM** (Document Object Model): la rappresentazione ad albero del documento, con nodo radice, nodi intermedi (elementi) e foglie (testo, commenti); il DOM può essere modificato durante la vita della pagina e “muore” alla chiusura o al ricaricamento.
3. Appena ha le risorse necessarie, il browser **renderizza** la pagina (la disegna applicando gli stili). Se la pagina è passiva, il ciclo termina qui.
4. Appena riceve gli script, il browser **li esegue**. In teoria i passi 3 e 4 potrebbero avvenire in qualsiasi ordine; in pratica si fa in modo che gli script girino quando il resto della pagina è caricato. Gli script predispongono l'ambiente di esecuzione: creano oggetti che mantengono lo stato e *registrano* le funzioni da eseguire in corrispondenza degli eventi. Due tipologie principali di eventi: **eventi utente** (click, tasti, scrolling, tap) ed **eventi di rete** (arrivo della risposta a una richiesta HTTP, timeout).
5. **Sopraggiunge un evento** (utente o di rete): il browser attiva la funzione registrata. I suoi effetti “percepibili” possono essere: generare altre connessioni HTTP (per dati o per spostarsi di pagina) e/o modificare l'aspetto della pagina. Le richieste HTTP generate *non sono bloccanti*: vengono avviate, e la gestione della futura risposta è demandata in modo asincrono a una funzione registrata nell'ambiente di esecuzione.
6. Quando l'aspetto della pagina viene modificato, il browser **ri-renderizza solo le parti modificate**: è questo a rendere l'interazione fluida. La ri-renderizzazione avviene appena l'ambiente JavaScript rilascia il controllo sul thread di esecuzione.

Il ciclo prosegue (eventi → modifiche → re-rendering) finché la pagina resta aperta.

1.8 Il protocollo HTTP in dettaglio

Definizione – Struttura di richieste e risposte HTTP

Richieste e risposte HTTP condividono la stessa struttura generale:

- **Start line**: definisce la natura del messaggio (*request line* / *response line*);
- **Header**: coppie **nome**: **valore** che dettagliano il messaggio;
- una **riga vuota** separa le meta-informazioni (start line + header) dal
- **Body** (o *payload*): il contenuto vero e proprio.

1.8.1 La HTTP Request

- **Request line**: *HTTP method* (o verb: azione da eseguire sul server; i due “di base” sono **GET** – il server fornisce informazioni – e **POST** – il server elabora i dati che riceve), *destinazione* (URL completa o solo path assoluto, assumendo come hostname il ricevente), *versione* del protocollo.

- **Header:** molti sono impostati automaticamente dal browser. I più comunemente impostati dal programmatore: **Accept** (tipi di dati accettati in risposta), **Content-Type** (tipo dei dati nel body), **User-Agent** (informazioni sul client).
- **Body:** le GET non ce l'hanno (chiedono dati, non ne inviano). Le POST sì, tipicamente di tipo `application/x-www-form-urlencoded` (dati di un form inviato dal browser) o `application/json` (dati elaborati e inviati da una app client-side).

Definizione – Tipi MIME

Negli header HTTP il tipo di dato si esprime nel formato **MIME** (Multipurpose Internet Mail Extension, nato per gli allegati email): `tipo/sottotipo`. Esempi comuni: `text/html`, `text/plain`, `text/css`, `text/javascript`, `image/jpeg`, `image/png`, `application/json`, `application/pdf`, `application/xml`, `video/mp4`.

1.8.2 La HTTP Response

- **Response line:** *versione*; *status code* (esito della richiesta, in 5 categorie identificate dalla prima cifra – in particolare **2XX** = successo, **4XX** = fallita per errore del *client*, **5XX** = fallita per errore del *server*); *status text* (descrizione testuale, es. code 404 → “Not Found”).
- **Header** più comuni: **Content-Type**, **Content-Length** (lunghezza del body in byte), **Server** (tipologia di Web Server), **Date** (giorno e ora di invio).
- **Body:** le risposte lo hanno *necessariamente*. Tipi più comuni: `text/html` (pagina da visualizzare), `application/json` (dati per un'app client-side), e in generale `text/*`, `image/*`, `audio/*`, `video/*`, `application/*` (fogli di stile, script, media, file scaricabili...).

1.9 Applicazioni web “full stack”

Il caso più completo: applicazione **client-side** + **server-side**.

1. **Il Web Server è sempre online:** pubblica staticamente la pagina HTML con gli script dell'app client-side e mantiene in esecuzione l'app server-side (meccanismi del Web Server la “bootstrappano” all'avvio). Il Web Server è l'*ambiente operativo* dell'app server-side (come il S.O. lo è per le app desktop). L'insieme di URL su cui l'app server-side risponde è detto **API** dell'applicazione server-side.
2. **Il browser richiede la pagina HTML** dell'app client-side; seguono le transazioni per media e script. Poi: renderizza la pagina e attiva l'ambiente JavaScript eseguendo gli script (= avvio dell'app client-side). La memoria dinamica dell'ambiente *non* viene resettata finché la pagina resta aperta; ogni pagina ha un ambiente indipendente.
3. **L'utente genera eventi:** se all'evento è associata una funzione, viene eseguita. Effetti possibili (anche entrambi): generare una richiesta verso l'app server-side (di cui l'app client deve conoscere le URL) e/o modificare la pagina.
4. **Il browser effettua una richiesta HTTP** su una URL dell'API server-side (per ottenere dati, elaborare, salvare). La richiesta è *non bloccante* e restituisce subito il controllo. Le app client-side sono **single-thread** (esistono i Web Workers per thread in background, ma sono usati in casi particolari): è importante che le elaborazioni in risposta a eventi siano rapide per non dare la sensazione di interfaccia bloccata.
5. **Il Web Server inoltra la richiesta all'app server-side**, che la “spacchetta” (method, path, query string, body con **Content-Type**) per capire quale funzionalità attivare e con quali dati. Quasi sempre l'elaborazione interagisce con un **Database Server** (per ottenere informazioni da inviare o memorizzare dati). Il risultato viene confezionato in una risposta HTTP (body + **Content-Type** corretto + informazioni per abbinarla alla richiesta originale).

A differenza del client, il server gestisce *molte richieste in parallelo*: la maggior parte dei Web Server genera un **thread separato per ogni richiesta**; l'approccio alternativo (adottato da Node.js) è un **single-threaded loop** con elaborazioni non-bloccanti (asincrone).

6. **La risposta arriva al client**: il browser la traduce in un evento di rete e richiama la funzione registrata al momento della richiesta. Anche qui: possibile nuova richiesta e/o modifica della pagina.
7. **Il browser ri-renderizza** le sole parti modificate, appena il thread JavaScript rilascia il controllo.

1.10 Front-end e back-end vs client-side e server-side

Definizione – Architettura a strati

L'**architettura a strati** è un pattern di progettazione in cui la logica applicativa è suddivisa su più *strati* (layer), ognuno dei quali offre funzionalità allo strato superiore sfruttando quelle dello strato inferiore. Centrale è il concetto di **interfaccia** fra strati: lo strato che offre servizi definisce formalmente le richieste accettate (input) e le risposte (output). Vantaggi: ogni strato è gestibile separatamente (si può cambiarne l'implementazione senza toccare gli altri, se l'interfaccia resta stabile); sullo stesso strato possono convivere moduli alternativi con la stessa interfaccia.

- **Back-end** = lo strato *più basso*: gestisce la **persistenza dei dati**. Nelle app web tipicamente interagisce con un database server: traduce i dati dal formato dell'applicazione a quello del DB (es. record di tabelle relazionali) e viceversa; può occuparsi di verifiche sui dati, transazioni, rollback. È la *source of truth* per il Data Model.
- **Front-end** = lo strato *più alto*: si occupa dell'**interazione con l'utente**. Ne fanno parte la User Interface (l'interfaccia verso lo "strato superiore", che è un essere umano!) ed eventuali moduli che rendono fruibili le informazioni.
- **Strati intermedi**: elaborazioni che non appartengono pienamente né all'uno né all'altro (eventualmente con servizi di terze parti); ha senso isolarle se si vuole renderle indipendenti da entrambi.

Da ricordare – Due distinzioni diverse

Client-side / server-side è una distinzione sul *luogo di esecuzione*: client-side = codice eseguito dal Web Client (browser, col suo motore JavaScript); server-side = codice eseguito dentro il Web Server (con il suo agente di esecuzione, es. JVM). **Front-end / back-end** è una distinzione sul *ruolo/responsabilità*: front-end = interfaccia con l'utente (presentare informazioni, raccogliere azioni); back-end = logica applicativa e gestione/persistenza dei dati.

Perché non coincidono. Nelle app web il front-end è *necessariamente* in parte lato client (è l'unico che può avere una UI) e il back-end è *tipicamente* (ma non obbligatoriamente) lato server. Gli strati intermedi possono stare da entrambe le parti: le elaborazioni lato client non sovraccaricano il server, ma richiedono che i dati necessari siano sul client (problemi: riservatezza, volumi di dati); quelle lato server accedono a tutto il DB, ma elaborazioni costose creano problemi di scalabilità. La configurazione tipica (front-end lato client + back-end lato server) fa sì che i termini vengano spesso usati come sinonimi, ma **una distinzione riguarda l'organizzazione logica, l'altra quella implementativa**: ad esempio la generazione dinamica di pagine HTML lato server (CMS come Wordpress o Moodle) è logica di *front-end* eseguita *server-side*.

2. Il front-end: HTML

2.1 Il browser come ambiente di esecuzione

Il client-side di una web app è costituito da: una o più **pagine HTML**, ciascuna corredata da uno o più **fogli di stile CSS** e associata a zero o più **moduli JavaScript** (una pagina senza JS è passiva; in un'app multi-pagina alcune pagine possono essere di sola lettura).

Da ricordare – Sandbox

Il browser vede *ogni pagina come a sé stante*: ciascuna viene eseguita in una “scatola chiusa” (**sandbox**) creata al caricamento. Se la pagina viene ricaricata, la sandbox viene distrutta e ricreata: **la memoria dell'applicazione si perde**. Si può ovviare parzialmente con i meccanismi di *storage locale* del browser, condivisibili fra pagine della stessa *origine* (stesso server), ma con diversi limiti. Se il client-side ha più pagine, è compito del *server-side* gestire il flusso di informazioni fra una pagina e l'altra.

Ricevuti HTML + CSS + JS, il browser usa HTML e CSS per inizializzare la *vista*:

- il **documento HTML** definisce la *struttura logica* e la *semantica* delle parti della pagina (sezioni, menu di navigazione, ruolo dei testi, media, elementi di input, collegamenti ipertestuali);
- i **fogli CSS** sono insiemi di regole che definiscono l'*aspetto grafico* degli elementi (caratteri, colori, dimensioni, collocazione);
- con HTML e CSS il browser inizializza il **DOM** (Document Object Model): la struttura dati che mantiene in memoria tutte le informazioni sulla visualizzazione; può essere modificata durante la vita della pagina, che termina alla chiusura o al ricaricamento.

2.2 HTML: un linguaggio di annotazione

HTML descrive struttura e aspetto di un documento multimediale e ipertestuale usando solo testo semplice (codifica UTF-8) e le URL dei media da includere. Ogni **elemento** è caratterizzato da:

- un **tipo**;
- un **contenuto** (testo e/o altri elementi; alcuni tipi non lo prevedono);
- una serie di **attributi** opzionali, dipendenti dal tipo.

La *specifica formale* di HTML definisce i tipi esistenti, il loro “significato inteso”, il contenuto atteso e gli attributi possibili. Il browser usa la strutturazione (ad albero) per organizzare la visualizzazione.

Esempio – Sintassi di un elemento

```
<XYZ attrib=val> contenuto </XYZ>
```

Tag di *apertura* (nome del tipo fra parentesi angolari), *contenuto* (testo e/o altri elementi), tag di *chiusura* (uguale ma con /). Gli attributi si indicano nel tag di apertura. Attributi non previsti dalla specifica sono ignorati dal browser, ma funzioni JavaScript possono sfruttare attributi *custom* definiti dal programmatore, che per convenzione hanno nome della forma `data-*`.

2.2.1 Struttura di un documento

```
<!DOCTYPE html>
<html lang="en">
  <head> ... </head>
  <body> ... </body>
</html>
```

- **Preambolo** `<!DOCTYPE html>`: dichiara la versione della specifica; esiste per retrocompatibilità ed è di fatto sempre uguale.
- **HEAD** (intestazione): *meta-informazioni* sul documento, usate dal browser ma non visualizzate come contenuti.
- **BODY** (corpo): il contenuto effettivo, ciò che viene visualizzato.

2.2.2 Elementi a blocco e in linea

In prima approssimazione gli elementi si dividono in **elementi a blocco** (definiscono un riquadro) ed **elementi in linea** (seguono il fluire del testo). In un documento ben strutturato gli elementi a blocco stanno solo dentro altri elementi a blocco: un blocco dentro un elemento in linea porta a comportamenti imprevisti. Il browser “fa del suo meglio” per visualizzare comunque, ma specie progettando UI serve una visualizzazione prevedibile e controllabile, non affidata alle strategie compensative del browser.

2.3 Collegamenti ipertestuali

```
<a href="https://it.wikipedia.org/wiki/...">testo cliccabile</a>
```

- L'elemento è **A** (*anchor*); può contenere sia elementi in linea che a blocco. Il contenuto è lo **hotspot**: ciò su cui l'utente clicca.
- La destinazione (*target*) è nell'attributo **href**, una URL valida: **esterna** (specifica dominio e path, dominio diverso dall'attuale), **assoluta** (stesso dominio, path completo dalla radice, inizia con /), **relativa** (stesso dominio, path relativo al documento attuale).

2.4 Gli elementi principali

2.4.1 Testo

Elementi *a blocco*:

- **P**: paragrafo.
- **H1...H6**: titoli a importanza decrescente.
- **UL**: lista a punti; contiene elementi **LI**. Attributo **type** (`circle`, `disc`, `square`) per il tipo di punto.
- **OL**: lista numerata di **LI**. **type** (`a`, `A`, `i`, `I`, `1`) per numerazione alfabetica/romana/intera; **start** per il valore iniziale.

Elementi *in linea*:

- **EM**: testo da enfatizzare (default corsivo).
- **STRONG**: enfasi forte (default grassetto).
- **SPAN**: testo da distinguere; il browser di default non lo distingue, l'aspetto si stabilisce poi via CSS.

2.4.2 Tabelle

Per contenuti realmente tabellari (dati in righe/colonne).

Attenzione

Non usare le tabelle per il layout della pagina: per la disposizione dei contenuti esistono gli elementi strutturali in combinazione con i CSS.

- **TABLE:** l'intera tabella; contiene righe.
- **TR:** una riga; contiene celle.
- **TD:** una cella. Attributi `rowspan/colspan` (interi > 0 , default 1) per celle che spaziano su più righe/colonne; la somma dei colspan dovrebbe essere costante per riga.
- **TH:** cella di intestazione; `scope = row o col`.
- **THEAD, TBODY, TFOOT:** raggruppano le righe in intestazione, corpo, piè.

2.4.3 Contenuti multimediali

- **IMG:** immagine, elemento *in linea*. `src` = URL (esterna, assoluta o relativa); `width/height` consigliati, così il browser imposta la pagina senza attendere il download.
- **AUDIO:** player audio incastonato (*embedded*); `src`, `autoplay` (true/false), `controls` (mostrare i comandi).
- **VIDEO:** analogo, con anche `width/height`.

Attenzione – Ridimensionare non riduce il traffico

L'immagine viene comunque scaricata *nella dimensione originale* e solo dopo ridimensionata dal browser. Se l'obiettivo è limitare il traffico di rete, servono più versioni dell'immagine sul server, usando la URL di quella della dimensione voluta.

2.4.4 Elementi strutturali

L'uso varia fra pagine “redazionali” (documenti, articoli, blog: interazione minima) e “applicative” (la pagina è la UI di un'applicazione: utente attivo).

- **DIV:** suddivisione strutturale *senza significato* a priori; fondamentale per dare un layout preciso, quasi indispensabile nelle UI.
- **SECTION:** suddivide la pagina in sezioni indipendenti.
- **ARTICLE:** contraddistingue un articolo presentabile anche indipendentemente dal resto.
- **NAV:** area dedicata a un menu di navigazione.
- **HEADER:** intestazione *visibile* di pagina/sezione (titolo, logo). **Da non confondere con HEAD**, che contiene la meta-intestazione non visualizzata.
- **FOOTER:** piè di pagina (contatti, cookie, ecc.).

2.5 I moduli (form) HTML

L'HTML puro permette moduli con cui l'utente invia dati al web server (sensato solo se il server li riceve con una app in grado di elaborarli). Qui parliamo di form *gestiti dal browser*; vedremo poi form gestiti da JavaScript, più flessibili.

2.5.1 Il tag FORM

Definisce il comportamento del modulo tramite gli attributi:

- **action:** la URL (esterna, relativa o assoluta) a cui inviare i dati;
- **method:** il method HTTP. Con **GET** i dati vanno codificati come *query string* nella URL; con **POST** vanno nel *body* con **Content-Type:** `application/x-www-form-urlencoded`.

In entrambi i casi i dati hanno la forma `nome1=valore1&nome2=valore2&...&nomeK=valoreK`: i nomi vengono dagli elementi di input, i valori da ciò che l'utente ha inserito.

2.5.2 Gli elementi di input

Dentro un FORM può esserci qualsiasi HTML; per raccogliere dati servono elementi di input, per lo più col tag **INPUT** e il suo attributo `type`.

Attenzione

Tutti gli INPUT richiedono *obbligatoriamente* l'attributo **name**: è il nome del campo codificato nella richiesta HTTP.

- `<input type="text"/>`: casella di testo. Attributi: `size` (larghezza in caratteri), `value` (valore preimpostato modificabile), `placeholder` (indicazione che *non* conta come valore e scompare digitando).
- `<input type="checkbox"/>`: `value` = valore riportato se contrassegnata; `checked` = inizialmente selezionata.
- `<input type="radio"/>`: i radio-button di uno stesso gruppo devono avere lo *stesso* `name` (solo uno selezionabile); `value` identifica quale è stato scelto; `checked` per la selezione iniziale.
- `<input type="submit" value="Premi qui"/>`: **obbligatorio** in un form HTML puro; non fornisce un dato, ma provoca l'invio secondo `action` e `method`; `value` è il testo sul pulsante.
- `<input type="reset" value="Ricomincia"/>`: opzionale; riporta tutti gli input allo stato iniziale.

Altri due elementi comuni nei form:

- **LABEL**: etichetta esplicativa associata a un INPUT (essenziale per checkbox e radio). L'INPUT deve avere un `id`; lo stesso valore va nell'attributo `for` della LABEL: `<label for="inputid">...</label>`.
- **SELECT**: menu a tendina; `name` sta sul SELECT. Le opzioni sono elementi **OPTION** al suo interno, ciascuna con il proprio `value` (riportato nella richiesta in base alla scelta):

```
<select name="scelta">
  <option value="opt1">Prima opzione</option>
  <option value="opt2">Seconda opzione</option>
</select>
```


3. Il front-end: CSS

3.1 Le regole di stile

Definizione – Regola CSS

Lo stile grafico di un documento HTML è descritto da **regole** raccolte in file **.css**. Una regola è composta da un **selettore** (filtra gli elementi a cui la regola si applica) e da un **blocco di dichiarazioni** fra graffe; ogni **dichiarazione** è della forma **proprietà: valore;**.

```
h1 {  
  background-color: darkslategrey;  
  color: bisque;  
  padding: 10px;  
  font-family: sans-serif;  
}
```

La specifica CSS definisce un lunghissimo elenco di proprietà (e per ognuna i tipi di valori ammessi).

3.1.1 Assegnare regole di stile a un HTML

- **Foglio esterno:** documento separato, collegato nello HEAD con `<link rel="stylesheet" href="styles/style.css">`.
- **Foglio interno:** regole scritte nello HEAD dentro un elemento `<style>...</style>`.
- **Stile in linea:** dichiarazioni associate a un singolo elemento tramite l'attributo `style`, es. `<h1 style="color: red; font-size: 15pt;">`.

3.1.2 Regole a cascata

Cosa succede se a uno stesso elemento si applicano regole con impostazioni contrastanti (stessa proprietà, valori diversi)? Tre meccanismi:

- **Cascata:** conta l'*ordine* in cui il browser riceve le regole – a parità di specificità vince quella definita *per ultima*. Conta l'ordine dei fogli esterni; i fogli interni vengono dopo gli esterni; le dichiarazioni in linea per ultime.
- **Specificità:** quanto *precisamente* il selettore individua l'elemento (calcolata con un algoritmo di cui non vediamo i dettagli). Le dichiarazioni più specifiche in assoluto sono quelle *in linea*.
- **Ereditarietà:** alcune proprietà vengono ereditate di default dal genitore, o i loro valori sono calcolati a partire da esso. Non influenza l'algoritmo di scelta, ma ha effetto sul risultato – da tenere presente quando i CSS “non si comportano come vorremmo”.

3.2 Proprietà e valori

Due macro-famiglie di proprietà molto usate:

- **tipografiche** (caratteri e testo): `font-size`, `font-family`, `font-weight`, `font-style`, `text-decoration`, `line-height`, `color`...

- **del riquadro** (box) che delimita l'elemento: `width`, `height`, `max-/min-width`, `max-/min-height`, `padding`, `margin`, `border`, `border-radius`, `background-color`, `position`, `top/-left/-bottom/-right...` (la libertà effettiva dipende dalla modalità di *display*: alcune dispongono i riquadri con un algoritmo e ignorano in tutto o in parte queste indicazioni).

Riferimenti completi: MDN (developer.mozilla.org) e W3Schools.

3.2.1 Tipi di valori

- **Valori predefiniti** (costanti, tipo enumerativo): `font-weight: bold`; `border: solid`; `color: red`; `margin: auto`;
- **Valori metrici**: numero + unità di misura. Unità *absolute* (`px`, `mm`, `in`, `pt...`) o *relative*, in proporzione a dimensioni dell'elemento radice o del contenitore: `font-size: 2rem`; (doppio della dimensione carattere della radice), `width: 50%`; (metà del contenitore), `max-width: 50vw`;...
- **Colori**: RGB o RGBα: `#dfff76`, `rgb(255,0,106)`, `rgba(255,0,106,0.1)`.

3.2.2 Proprietà composite

Alcune proprietà sono scorciatoie per impostarne più d'una insieme. Esempio con i margini:

```
margin: 5px 10px 0 20px; /* top - right - bottom - left */
margin: 10px;           /* tutti e 4 uguali */
margin: 10px 20px;      /* verticali - orizzontali */
```

3.3 I selettori

I selettori sono vere e proprie *query sul documento HTML*: è la loro flessibilità a rendere potente il CSS. Si dividono in semplici, composti e complessi.

3.3.1 Selettori semplici

- **per TAG**: coincide con un tag HTML, seleziona tutti gli elementi di quel tipo. Es. `img {...}`
- **per CLASSE** (la categoria più usata): la classe si assegna con l'attributo `class`; le classi sono stringhe libere che *etichettano* gli elementi per raggrupparli e selezionarli (nei CSS e in JavaScript). Selettore: punto + nome, es. `.articolo {...}`
- **per ID**: id univoco assegnato con l'attributo `id`; selettore: `#` + id, es. `#utente`. Meno usati delle classi, soprattutto per la difficoltà di garantirne l'univocità.
- **per ATTRIBUTO**, fra parentesi quadre: `[attr]` (elementi che dichiarano `attr`, con qualunque valore) oppure `[attr OP "val"]`, dove il significato di “corrispondere” dipende dall'operatore: `[attr="val"]` uguaglianza esatta, `[attr*="text"]` il valore *contiene* la stringa.
- **per PSEUDOCLASSE**: raggruppamenti predefiniti determinati dal browser in base allo *stato* dell'elemento, dinamico (`:hover` – mouse sopra) o statico (`:first-child` – primo figlio). Selettore: due punti + nome.
- **UNIVERSALE**: `*`, seleziona tutti gli elementi.

3.3.2 Selettori composti

Si ottengono *giustapponendo* (senza spazi) più selettori semplici; l'effetto è un **AND**: sono selezionati gli elementi che rispettano *tutti* i selettori. Vincoli: al più *un* selettore per tag (o l'universale),

e se presente deve essere il *primo* della composizione (l’universale come primo può anche essere omissso). L’ordine non cambia il significato, ma può influire sulla specificità.

Esempio

```
div.importante = tutti i DIV di classe importante; label[name="anno"] = tutte le LABEL
con name pari ad "anno"; .elenco.attivo = elementi (di qualsiasi tipo) con entrambe le classi;
input[type="button"]:hover = i pulsanti, ma solo al passaggio del mouse.
```

3.3.3 Selettori complessi

Selezionano in base alla *posizione nella struttura* del documento: sequenze di selettori semplici/-compositi separate da **combinatori**. Forma: **S comb A** = “seleziona gli elementi che rispettano A e che sono nella relazione **comb** con un elemento che rispetta S”.

- **spazio** = *discendente di*: `.articolo h1` = tutti gli H1 dentro (a qualsiasi profondità) un elemento di classe `articolo`.
- **>** = *figlio di*: `ul.menu > li` = tutti gli LI direttamente dentro un UL di classe `menu`.
- **~** = *fratello successivo di*: `.introduzione ~ div` = tutti i DIV che, nello stesso genitore, vengono (anche non consecutivamente) dopo un elemento di classe `introduzione`.
- **+** = *prossimo fratello di*: `h1 + p` = il P che, nello stesso genitore, viene *immediatamente* dopo un H1.

3.3.4 Liste di selettori

Una regola può applicarsi a una *lista* di selettori separati da virgole: è un **OR** (selezionati gli elementi che rispettano *almeno uno* dei selettori).

```
h2, .capitolo > h1, input[type="text"]:hover { /* stili */ }
```

3.4 Il layout: display

La disposizione degli elementi è stabilita dal browser in base a due fattori:

1. la **struttura della pagina**: un modello a *scatole cinesi* in cui ogni elemento è contenuto in un altro (tranne la radice `html`); idealmente ogni elemento è posizionato dentro il riquadro del genitore. La struttura a scatole è anche una struttura *ad albero* (fondamentale per il DOM);
2. lo stile di **display** di ciascun elemento, espresso da due proprietà:
 - **display-inside**: *come l’elemento dispone i propri contenuti*;
 - **display-outside**: *come l’elemento preferirebbe essere disposto nel contenitore*.

Ogni tipo di elemento ha valori di default, modificabili via CSS. Alcuni display-inside tengono conto del display-outside dei figli, altri lo ignorano.

La proprietà **display** è una scorciatoia per entrambi i valori (prima l’outside, poi l’inside); si può esprimere anche uno solo dei due:

```
p { display: inline flow; } /* outside: inline, inside: flow */
p { display: block; } /* outside block, inside resta flow */
p { display: flex; } /* inside flex, outside di default */
```

3.4.1 display-inside: flow

È il **default** di (quasi) tutti gli elementi: dispone i contenuti come in un articolo di rivista. Qui è *rilevante il display-outside dei contenuti*:

- un elemento interno con outside **block** ottiene un riquadro che occupa *tutto il contenitore in orizzontale* e in altezza quanto gli serve (come un paragrafo);
- un elemento con outside **inline** è trattato come una sequenza di testo/simboli che scorre da sinistra a destra su più righe; due inline consecutivi proseguono uno dopo l'altro sulla stessa riga.

Con la proprietà **float** (**left**/**right**) un elemento esce dal flusso delle righe, che gli scorrono intorno (come un'immagine “fluttuante” nel testo).

flow-root: variante di flow interessante su un elemento con outside *inline*: l'elemento si dispone inline nel contenitore ma *non* si distribuisce nel flusso: si comporta come un unico grosso “carattere”.

3.4.2 display-inside: flex

Dispone gli elementi come *sequenza di riquadri in riga o colonna*, adattandone le dimensioni. Si imposta con **display: flex** (o **inline flex** / **block flex**). **Ignora il display-outside dei contenuti**: a ognuno viene assegnato un riquadro. Proprietà del *contenitore*:

- **flex-direction**: **row** / **row-reverse** / **column** / **column-reverse**;
- **flex-wrap**: se andare a capo quando lo spazio non basta (con **nowrap** i contenuti “sbordano”);
- **justify-content**: allineamento lungo la direzione principale – **flex-start** (sinistra, o alto se in colonna), **flex-end**, **center**, **space-between** (spazio in eccesso *fra* gli elementi), **space-around** (spazio distribuito *attorno* agli elementi);
- **align-items**: allineamento sull'asse trasversale – **flex-start** (alto, o sinistra se in colonna), **flex-end**, **center**, **stretch** (allunga a riempire), **baseline** (linea di base del testo).

Proprietà degli *elementi contenuti*:

- **flex-grow** (≥ 0 , default 0): se, e in che proporzione, l'elemento è disposto ad *allargarsi* per occupare lo spazio disponibile;
- **flex-shrink** (≥ 0 , default 1): se, e in che proporzione, è disposto a *restringersi* per non eccedere lo spazio.

3.4.3 display-inside: grid

Definisce una **griglia** (righe $\times$ colonne); ogni elemento occupa una o più celle contigue. Si imposta con **display: grid** (o **inline grid** / **block grid**); anch'esso *ignora il display-outside dei contenuti*.

- La griglia si configura sul contenitore con **grid-template-rows** e **grid-template-columns**, che elencano altezze e larghezze; l'unità **fr** (*frazione*) esprime dimensioni proporzionali; **repeat(n, v)** è una scorciatoia:

```
grid-template-rows: 50px 1fr 2fr;  
grid-template-columns: repeat(4, 1fr) 10vw;
```

- Ogni elemento interno dichiara le celle che occupa con `grid-row-start`, `grid-row-end`, `grid-column-start`, `grid-column-end`; `end` può usare la forma `span N` (“estenditi per N celle”):

```
grid-row-start: 2;    grid-row-end: span 3;  
grid-column-start: 2; grid-column-end: span 2;
```

- Ulteriori proprietà regolano gli allineamenti reciproci e dentro l’area assegnata.

3.4.4 `display: none`

La proprietà `display` può valere `none`: l’elemento viene **rimosso dalla visualizzazione**.

3.5 Strumenti per CSS

- **SCSS / SASS**: SCSS (*Sassy CSS*) è un linguaggio più evoluto per descrivere regole di stile; SASS è il *pre-processor* che traduce `.scss` in `.css` (il browser interpreta solo quest’ultimo). Vantaggi: costanti riutilizzabili, inclusione di fogli in altri fogli, regole annidate (che generano automaticamente composizioni di selettori).
- **Bootstrap** (v5.3): insieme di CSS e JavaScript con stili e comportamenti di default per i componenti comuni delle UI. Uno stile predefinito è semplicemente una *classe CSS*: usare Bootstrap = conoscere le sue classi e usarle nel proprio HTML; si può usare solo la parte CSS o anche i componenti predefiniti (bottoni, tendine, finestre modali). Vantaggi: meno codice per gli elementi frequenti, UI *responsive* più semplici. Svantaggi: UI “fatte con lo stampino”, customizzazione non banale.
- **Tailwind**: *rovescia* il rapporto HTML/CSS. Tradizionalmente l’HTML esprime i contenuti caratterizzandoli con classi semantiche e il CSS dipende dall’HTML; in Tailwind il CSS definisce *classi di utilità* generiche (es. `bg-white`, `font-semibold`) ed è l’HTML a dipendere dal CSS. Il CSS diventa estremamente riutilizzabile; svantaggio significativo: HTML più verboso e ripetitivo (molto attenuato se si usano framework a componenti).

4. JavaScript: le basi

4.1 Carta d'identità del linguaggio

Definizione – Caratteristiche di JavaScript

- È un linguaggio **interpretato**: necessita di un motore di esecuzione (interprete).
- È **imperativo e strutturato**: programmi fatti di istruzioni, organizzabili in blocchi, con condizionali e cicli.
- È **debolmente tipato**: i dati vengono convertiti automaticamente da un tipo all'altro se il calcolo di un'espressione lo richiede (35 può diventare "35", **false** può diventare 0).
- È **dinamicamente tipato**: il tipo di un'espressione è stabilito solo a run-time; non si dichiara in anticipo il tipo di variabili, parametri o valori di ritorno.
- È **object-oriented basato su prototipi**: gli oggetti non hanno bisogno di classi per essere creati (un oggetto è una collezione di coppie proprietà/valore; un oggetto può fare da *prototipo* a un altro, che ne eredita le proprietà; le classi esistono comunque, per semplificare la creazione di oggetti con le stesse proprietà e lo stesso prototipo).
- Supporta la **programmazione funzionale**: le funzioni sono *tipi di dati* – possono essere create dinamicamente, memorizzate in variabili, passate come parametri, assegnate a proprietà di oggetti.

4.1.1 JavaScript o ECMAScript?

JavaScript nasce come linguaggio di scripting per il web; dopo un periodo “selvaggio” in cui ogni browser dava la propria interpretazione, si è imposta la standardizzazione **ECMAScript** (ES): la specifica ufficiale, in costante evoluzione. “JavaScript” è l'implementazione della specifica realizzata da Mozilla (Firefox); ormai tutti i browser implementano correttamente ECMAScript. Le feature aggiunte nelle edizioni successive servono a una programmazione più concisa e manutenevole, raramente introducono costrutti non replicabili nelle versioni precedenti: per questo un programma per una versione recente può essere *tradotto automaticamente* in una versione precedente. Questa traduzione, detta **transpilazione**, è diffusa per far girare i programmi anche su browser meno aggiornati. L'ultima edizione che ha modificato sostanzialmente il linguaggio è stata **ES2015 (ES6)**; il corso non usa nulla di posteriore a **ES2020 (ES11)**.

4.1.2 Linguaggi interpretati, compilati, ibridi

Le istruzioni sono eseguite da un *agente di calcolo*, che decide il set di istruzioni valide e il loro significato. L'agente di base è il processore (linguaggio macchina, inutilizzabile direttamente). I linguaggi di alto livello si dividono in:

- **compilati** (es. C): il programma viene tradotto *una volta per tutte* in linguaggio macchina da un *compilatore*; il risultato è un eseguibile, specifico per la famiglia di processori;
- **interpretati** (es. JavaScript): usano un agente di calcolo di livello più alto – l'*interprete* o *motore di esecuzione* (lui sì compilato) – che esegue direttamente le istruzioni. L'agente realizza una **macchina virtuale**: non esiste una macchina fisica che esegue quelle istruzioni, ma per chi programma è come se ci fosse;
- **ibridi** (es. Java): si compila in un linguaggio intermedio (Java ByteCode) che viene poi interpretato da un motore (la JVM). Lo schema può avere anche più livelli: si potrebbe

inventare un linguaggio “VoiceScript” con un motore scritto in JavaScript, il quale a sua volta gira su un motore JavaScript, che invoca istruzioni macchina.

4.2 Eseguire JavaScript nel browser

Il **motore di esecuzione** realizza un **ambiente di esecuzione**: “programma in esecuzione” + “memoria di esecuzione”. È un ambiente *vivo* (oggetti e variabili aggiunti, tolti, modificati) e può essere corredato di **API** che offrono costrutti per interagire con quell’ambiente. Nel browser: il motore crea una *sandbox* separata per ogni pagina, resettata al ricaricamento. L’ambiente contiene l’oggetto **document** (il DOM della pagina), la **DOM API** per consultarlo e modificarlo, e la **fetch API** per le richieste HTTP.

4.2.1 Collegare uno script a una pagina

```
<script type="application/javascript" src="path/to/myscript.js"></script>
```

Tipicamente in HEAD; lo script viene però eseguito *immediatamente*, senza aspettare il caricamento della pagina – un problema se interagisce con **document**, che potrebbe essere ancora vuoto. Tre modi per rimandare l’esecuzione:

1. spostare SCRIPT *al fondo*, dopo la chiusura di BODY (ma prima di quella di HTML);
2. usare l’attributo **defer**;
3. dichiarare **type="module"**: indica che il file è il *punto d’ingresso* di un’applicazione su più moduli. È la soluzione più usata.

4.3 Tipi, variabili e strutture di controllo

4.3.1 I tipi base

number, **boolean**, **string**, **object**, **undefined**, **null**, **bigint**, **symbol** (array e funzioni sono oggetti).

- Nessuna distinzione interi/decimali: tutti **number**, floating point a 64 bit (come i **double** di Java); **bigint** per interi molto grandi.
- Le stringhe (codifica UTF-16) si definiscono con virgolette `"`, apici `'` (equivalenti, salvo poter usare l’uno dentro l’altro senza escape) o **backtick** ```: queste ultime sono *template literal* – tutto ciò che sta fra i backtick fa parte della stringa, spazi e tabulazioni inclusi (e, come vedremo negli esempi, permettono l’interpolazione `${espr}`).
- **undefined** e **null**: ciascuno è un tipo con un unico valore. Entrambi rappresentano operazioni non andate a buon fine; **undefined** = “nessun valore” (variabile non assegnata, funzione che non restituisce nulla), **null** = “oggetto nullo”. In pratica usati in modo intercambiabile: meglio sceglierne uno e attenersi, o avere ben chiaro quando si usa l’uno e quando l’altro.

4.3.2 Variabili

```
let anno = 2024;      // dichiarazione, inizializzazione opzionale
const pi = 3.14;      // va inizializzata subito, non riassegnabile
```

Le variabili *non hanno tipo*: possono contenere valori di tipi diversi in momenti diversi.

Attenzione – `const`: costante $\neq$ immutabile

Il valore di una `const` è *costante* ma non *immutabile*: se contiene un tipo-riferimento (oggetto, array), il riferimento non cambia (stessa identità in memoria) ma si possono aggiungere/togliere elementi all'array o modificare le proprietà dell'oggetto.

4.3.3 Operatori e uguaglianza

Operatori aritmetici e booleani standard; `+` concatena anche stringhe (come Java). Poiché JS è debolmente tipato, `12 == "12"` risulta *vero*: per questo esistono `===` e `!==`:

$$(a === b) \Leftrightarrow (a == b \ \&\& \ \text{typeof } a == \text{typeof } b)$$

Attenzione

`==` e `!=` sono fonte di innumerevoli bug (es. `undefined` convertito a `false` o 0 maschera errori di calcolo): buona prassi usare *solo* `===` e `!==`.

4.3.4 Controllo di flusso

`if`, `while`, `do...while` identici a C/Java. Tre varianti di `for`:

- `for(init; condition; increment)`: il `for` classico;
- `for (const elem of iterabile)`: itera sugli *elementi* di una struttura iterabile (array, string sui caratteri, `Set`, `Map`);
- `for (const key in oggetto)`: itera sulle *proprietà* di un oggetto (`key` è il nome della proprietà come stringa).

4.4 Array

```
let amici = ["Antonio", "Paola", "Giorgio", "Andrea", "Ellen"];
let vuoto = [];
amici[2] // lettura/scrittura per indice
```

- Nessuna dimensione prestabilita: la lunghezza è ultima posizione occupata + 1.
- Possono essere *sparsi*: assegnando la posizione 100 le posizioni 5–99 restano `undefined`.
- Possono mescolare tipi diversi.

Gli array sono *oggetti* della classe `Array`, con proprietà e metodi:

- `a.length`: la lunghezza;
- `a.pop()`: restituisce, *rimuovendolo*, l'ultimo elemento;
- `a.push(v)`: aggiunge `v` in coda;
- `a.join(sep)`: stringa con gli elementi concatenati, separati da `sep`;
- `a.slice(start, end)`: *nuovo* array con gli elementi da `start` inclusa a `end` esclusa;
- `a.splice(start, delNum, add1, ..., addN)`: *modifica* l'array attuale da `start`: cancella `delNum` elementi e inserisce i nuovi; con solo cancellazione (nessun elemento elencato) o solo aggiunta (`delNum = 0`).

4.5 Oggetti base

Definizione – Object literal

Un oggetto è una collezione di coppie proprietà/valore. La forma più semplice è l'**object literal**: proprietà **prop**: valore separate da virgole, fra graffe.

```
const libro = {
  id: 8,
  titolo: "Pedagogia degli oppressi",
  autore: "Paulo Freire",
  stato: "borrowed"
}
```

I nomi devono essere ben formati (come le variabili: alfanumerici, \$, _; il primo carattere non può essere una cifra).

Accesso in lettura/scrittura con la **dot notation** (`libro.titolo`) o la **square bracket notation** (`libro["titolo"]`). Si possono assegnare nuove proprietà dopo la creazione (`libro.editore = "Neri Pozza";`); con le quadre si possono usare nomi non validi per la dot notation (`libro["ISBN-13"] = "...";` – contiene un trattino). Si possono anche cancellare: `delete libro.stato;`

4.6 Classi (introduzione)

```
class Studente {
  constructor(nome) {
    this.nome = nome;
    this.votiEsami = [];
  }
  aggiungiVoto(voto) { this.votiEsami.push(voto); }
}
const s = new Studente("Saverio");
s.aggiungiVoto(28);
```

- Il **costruttore** (`constructor`) inizializza le proprietà; l'oggetto in corso di inizializzazione è `this`.
- I **metodi** sono funzioni possedute da tutti gli oggetti della classe; `this` si riferisce all'oggetto su cui il metodo è invocato.
- Non c'è una sezione che elenca le proprietà: gli oggetti possono acquisirne/perderne dopo la creazione, e senza tipi l'elenco non darebbe vantaggi.
- Proprietà e metodi sono *tutti pubblici*; per l'“uso interno” si usa la convenzione dei nomi con underscore iniziale (non è un vincolo reale).
- **Ereditarietà**: `class Laureato extends Studente {...}`; il costruttore della sottoclasse deve invocare *per prima cosa* quello della superclasse con `super(...)`, poi imposta le proprietà aggiuntive. Gli oggetti della sottoclasse ereditano proprietà e metodi della superclasse.

4.7 Funzioni

Tre modi di dichiararle:

```
function myfun(x, y, z) { ...; return valore; } // named
const noname = function (x, y, z) { ...; return v; }; // anonymous
```

```
const arfun = (x, y, z) => { ...; return v; }; // arrow
const oneline = (x, y, z) => valore; // arrow, corpo-espressione
```

Attenzione

Invocando una funzione *non* è obbligatorio passare tutti i parametri: quelli mancanti valgono **undefined**. Una funzione senza **return** esplicita restituisce **undefined**. Quindi un’invocazione sbagliata o un **return** dimenticato producono errori che “saltano fuori” in punti molto diversi del programma.

Funzioni come dati. Le funzioni si possono dichiarare ovunque (visibili nello scope di creazione); si possono assegnare a variabili e a proprietà di oggetti. È importante distinguere:

- la *funzione in quanto oggetto* (indicata col suo nome, o col nome della variabile che la contiene): la funzione è un dato;
- l’*invocazione* (nome + parentesi tonde + parametri): un’espressione con un risultato (che può essere **undefined**... o un’altra funzione!).

Una funzione può essere passata come parametro a un’altra e invocata al suo interno col nome del parametro formale:

```
function applica(testo, analisi) { return analisi(testo); }
let contaparole = s => s.split(' ').length;
let n = applica("c'era una volta", contaparole);
```

Attenzione – Le proprietà-funzione non sono metodi

Assegnando una funzione a una proprietà di un oggetto *non* si ottiene automaticamente un metodo: se la funzione è una *arrow function*, durante l’invocazione **non è attivo il binding di this** (this non viene “vincolato” all’oggetto proprietario). Il tema del binding è approfondito nel capitolo sui prototipi.

4.8 Scope e closure

Definizione – Scope

Ogni blocco di istruzioni fra graffe istituisce uno **scope** (ambito di visibilità): blocchi **if/else**, **while/for**, corpi di funzione, o anche blocchi arbitrari. Gli scope possono essere *innestati*; il più esterno (l’intero script) è lo **scope globale**. La visibilità di variabili e nomi di funzione è limitata allo scope di definizione e ai suoi discendenti.

Definizione – Closure

Le variabili di uno scope **sopravvivono alla sua chiusura** fino a quando esiste uno scope interno attivo che le utilizza: questo meccanismo è detto **closure**. È tipico dei linguaggi funzionali ed è necessario perché le funzioni possano essere “elementi di prima classe”. In altri termini: la graffa chiusa segna la conclusione *statica* dello scope, ma l’esecuzione dell’ultima istruzione non ne segna la fine della *vita dinamica*.

Esempio

```
function functionCreator() {
  let somearray = ["1..", "2..", "3..", "Prova"];
  let somefun = (testo) => {
    for (let elem of somearray)
      console.log(`${elem} --> ${testo}`);
  }
  let update = (s) => { somearray.push(s); console.log("Aggiornato!"); }
  update("Via!");
  return somefun;
}
const risultato = functionCreator();
risultato("Eccomi!"); // somearray e' ancora "vivo"!
```

`somearray` è creato nello scope di `functionCreator` e usato da una funzione interna, che viene restituita e invocata *dopo* che `functionCreator` è terminata: grazie alla closure, `somearray` sopravvive al suo creatore.

4.9 Array e programmazione funzionale

Il fatto che le funzioni siano dati passabili come parametri permette uno stile **funzionale** e dunque *dichiarativo*: non si impartiscono istruzioni algoritmiche su *come* ottenere il risultato, si dichiara *cosa* si vuole ottenere.

Esempio – map: imperativo vs dichiarativo

```
// imperativo
const numeri = [1,5,8,12,13];
const quadrati = [];
for (let n of numeri) quadrati.push(n*n);
// dichiarativo
const quadrati2 = numeri.map(x => x * x);
```

`map` prende una funzione di mappatura (trasforma un singolo elemento) e restituisce un *nuovo* array con tutti gli elementi mappati.

Altri metodi “dichiarativi” degli array:

- `a.forEach(fun)`: esegue `fun` per ogni elemento (il valore di ritorno di `fun` è ignorato);
- `a.find(searchFun)`: primo elemento per cui `searchFun` (che restituisce boolean) è vera; esiste anche `findLast`;
- `a.findIndex(searchFun)`: come `find` ma restituisce la *posizione* (−1 se nessun elemento soddisfa); anche `findLastIndex`;
- `a.every(testFun)`: true se `testFun` è vera per *tutti*;
- `a.some(testFun)`: true se vera per *almeno uno*;
- `a.reduce(accumulatore, valoreIniziale)`: “accumula” i valori in un unico risultato; l’accumulatore è una funzione (valore accumulato, prossimo valore) → risultato dello step. Esempio, la somma: `numeri.reduce((somma, v) => somma + v, 0)`;

4.10 Moduli: organizzare il codice su più file

In JavaScript (a differenza di Java) il codice può essere diviso su più file in modo arbitrario. In teoria: uno script non viene compilato né controllato prima dell'esecuzione, quindi può usare funzioni/oggetti definiti altrove *purché* quei file siano stati eseguiti prima e l'ambiente sia popolato. Il modo rudimentale (ormai quasi solo in applicazioni molto vecchie) è includere gli script nell'HTML *nell'ordine giusto*: non regge per applicazioni complesse che usano estensivamente librerie di terze parti, dove è cruciale che le **dipendenze** siano chiare.

Definizione – Moduli

Ogni file JavaScript è un **modulo**. Un modulo può **importare** ed **esportare** oggetti, variabili, funzioni e classi.

4.10.1 Export

- **standard export**: si premette `export` alla dichiarazione:

```
export function pincopallino() {...}
export let nomeVar = ...
export class UnaClasse {...}
```

- **default export**: quando un modulo esporta *una sola* cosa; si premette `export default`. Ciò che non viene esportato resta *privato* del modulo – è anche un modo di proteggerne i meccanismi interni.

4.10.2 Import

- **default import**: `import nomeQualsiasi from "altroscrip.js"` – il nome è a scelta, tanto l'elemento esportato è uno solo;
- **standard import di base**: `import {cosa1, cosa2} from "altroscrip.js"` – si usano i nomi originali;
- **con renaming**: `import {cosa1 as thing1} from "altroscrip.js"` – si possono rinominare solo alcuni elementi;
- **con namespace**: `import * as Qualcosa from "altroscrip.js"` – importa tutto, con prefisso: `Qualcosa.x`.

5. JavaScript: prototipi, classi ed ereditarietà

5.1 I prototipi

Definizione – Prototipo

In JavaScript **ogni oggetto ha un prototipo**: un altro oggetto di cui *eredita le proprietà*. L'unico oggetto *privo* di prototipo è `Object.prototype`, che è il prototipo di default degli object literal.

Due oggetti “primordiali” formano la base di tutte le strutture dati del linguaggio:

- la **funzione Object**: invocata implicitamente ogni volta che c'è da creare un oggetto (es. dichiarando un object literal);
- l'oggetto **Object.prototype**: prototipo di default per tutti gli oggetti creati senza specificarne uno diverso. (La scrittura non deve stupire: in JS una funzione è un oggetto, e può quindi possedere proprietà.)

Da ricordare – Come funziona l'ereditarietà via prototipo

Quando si *legge* una proprietà di un oggetto (per conoscerne il valore o per invocare una proprietà-funzione), se il motore non la trova nell'oggetto stesso la cerca **nel suo prototipo**: per questo si dice che l'oggetto *eredita* le proprietà del prototipo. Il prototipo viene invece **ignorato in scrittura**: assegnando un valore a una proprietà inesistente, questa viene *aggiunta all'oggetto*. Se il prototipo aveva una proprietà con lo stesso nome siamo in presenza di **overriding**: l'erede sovrascrive (per sé) la proprietà dell'antenato – il motore troverà prima quella “locale” e ignorerà quella ereditata. Gli *altri* eredi continuano a condividere quella del prototipo.

Definizione – Prototype chain

Ogni oggetto ha un prototipo, che è a sua volta un oggetto con un prototipo, e così via fino a `Object.prototype`: questa è la **catena dei prototipi** (prototype chain). Prima di decidere che un oggetto *non* possiede una proprietà, il motore risale *tutta* la catena.

5.1.1 Accedere e modificare i prototipi

Si può accedere in ogni momento al prototipo di un oggetto tramite `oggetto.__proto__`, e attraverso di esso *modificarne le proprietà* – di fatto modificando ciò che è accessibile a **tutti** gli oggetti che fanno capo a quel prototipo (eliminando **abbaia** dal prototipo **fido**, anche **snoopy** e **cucciolo** la perdono). Vale anche per i prototipi predefiniti: aggiungendo una proprietà a `Object.prototype`, *tutti* gli oggetti dell'applicazione la acquisiscono – si può “plasmare o distruggere il linguaggio” a proprio rischio e pericolo.

È possibile (ma poco usato) bloccare le modifiche a un oggetto dopo la creazione:

- `Object.preventExtensions(ogg)`: non potrà acquisire nuove proprietà né cambiare prototipo;
- `Object.seal(ogg)`: *sigilla* – come sopra, ma non potrà nemmeno *perdere* proprietà;
- `Object.freeze(ogg)`: *congela* – oltre a sigillare, impedisce nuovi assegnamenti alle proprietà: di fatto read-only.

5.1.2 Creare oggetti a partire da un prototipo

```
const fido = {                                // prototipo di default: Object.prototype
  tipo: "Cane",
  nome: "Fido",
  abbaia: function () {}
}
const snoopy = Object.create(fido);           // fido come prototipo
snoopy.nome = "Snoopy";                      // overriding di "nome"
snoopy.razza = "Beagle";                     // proprietà originali
const cucciolo = Object.create(fido);
cucciolo.razza = "Jack Russel Terrier";
// cucciolo.nome vale "Fido" finché non lo ridefinisce
```

5.1.3 Determinare le proprietà di un oggetto

- `Object.keys(ogg)`: array dei nomi delle proprietà definite *nell'oggetto stesso*;
- `for (let key in ogg)`: cicla su *tutte* le chiavi valide, **incluse quelle della prototype chain** ma *escluse* quelle “di sistema” (es. le proprietà definite in `Object.prototype`);
- `ogg.hasOwnProperty("p")`: metodo definito in `Object.prototype` (dunque disponibile a tutti): *true* se l'oggetto *definisce* la proprietà; *false* se non esiste *o è ereditaria*.

5.2 Metodi e binding di `this`

Definizione – Metodo

Un **metodo** è una funzione *collegata* (bound) a un oggetto, che diventa il “soggetto” della funzione: nell'esecuzione, `this` si riferisce a quell'oggetto-soggetto.

Da ricordare – Quando avviene il binding

- Il binding avviene **implicitamente** per le proprietà-funzione definite come *named* o *anonymous function*.
- **Non avviene** per le proprietà-funzione definite come *arrow function*: una arrow assegnata a una proprietà *non* è un metodo (dentro di essa `this` non è l'oggetto proprietario).
- **Binding esplicito**: `f.bind(ogg)` crea una *copia* della funzione `f` bound all'oggetto specificato. Lo stesso effetto si ottiene assegnando la funzione a una proprietà dell'oggetto – a meno che non sia una arrow, nel qual caso il binding potrà essere solo esplicito.

```
const fido = {
  tipo: "Cane", nome: "Fido",
  abbaia: function () {}, // metodo (binding implicito)
  corre: () => {}          // NON metodo (arrow: niente binding)
}
function log(msg) { console.log(msg + ": " + this.nome); }
const stampaFido = log.bind(fido); // binding esplicito
fido.stampa = log;                 // binding implicito
```

5.3 I costruttori: creare oggetti “con lo stampino”

Il sistema originario per creare oggetti simili in serie sono i **costruttori**: *named function* (per convenzione con l’iniziale maiuscola) invocate premettendo la keyword **new**.

```
function Cane(nome, razza) {  
  this.tipo = "Cane";  
  this.nome = nome;  
  if (razza !== undefined) this.razza = razza;  
}  
const fido = new Cane("Fido");  
const snoopy = new Cane("Snoopy", "Beagle");
```

Da ricordare – Cosa succede con new Fun(...)

1. viene creato un oggetto da usare come *prototipo*, memorizzato in `Fun.prototype`;
2. viene creato un nuovo oggetto `x` con prototipo `Fun.prototype`;
3. viene realizzato il *binding* fra il nuovo oggetto e la funzione: `this` dentro la funzione si riferisce a `x`.

Dunque **tutti gli oggetti creati con new Fun(...)** condividono il medesimo prototipo.

5.4 Classi = costruttore + prototipo condiviso

Definizione – Classe in JavaScript

Il concetto di costruttore è alla base del concetto di classe: *due oggetti sono della stessa classe se creati dallo stesso costruttore*. Oggetti creati dallo stesso costruttore sono inizializzati allo stesso modo (stesse proprietà, ognuno coi propri valori: la proprietà è *dell’oggetto*, non del prototipo) e **condividono il prototipo**: per questo si possono dare “metodi alla classe” semplicemente aggiungendoli al prototipo.

Esempio – Il costrutto class è zucchero sintattico

Le due forme seguenti sono equivalenti: la “classe” non è altro che un costruttore, e i metodi scritti “dentro” la classe vengono assegnati al prototipo associato al costruttore.

```
// con costruttore e prototipo esplicito  
function Cane(nome, razza) {  
  this.tipo = "Cane"; this.nome = nome;  
  if (razza !== undefined) this.razza = razza;  
}  
Cane.prototype.abbaia = function() {}  
  
// con il costrutto class  
class Cane {  
  constructor(nome, razza) {  
    this.tipo = "Cane"; this.nome = nome;  
    if (razza !== undefined) this.razza = razza;  
  }  
  abbaia() {}  
}
```

5.4.1 Ereditarietà fra classi e catena dei prototipi

Con `class Beagle extends Cane`, la catena che si forma è:

`snoopy → Beagle.prototype → Cane.prototype → Object.prototype`

```
class Cane {
  constructor(nome, razza) { ... }
  abbaia() {}
}
class Beagle extends Cane {
  constructor(nome) {
    super("Beagle"); // prima cosa: costruttore della superclasse
    ...
  }
  scrivi() {}
}
const snoopy = new Beagle("Snoopy");
// snoopy trova scrivi() in Beagle.prototype,
// abbaia() risalendo a Cane.prototype
```


6. JavaScript e il DOM

6.1 Il Document Object Model

Definizione – DOM

Il **DOM** (Document Object Model) è la struttura dati che rappresenta il documento HTML e mantiene in memoria tutte le informazioni sulla sua visualizzazione. È un **albero**: nodo *root*, nodi intermedi (tipicamente elementi HTML) e foglie (testo, commenti). Viene inizializzato da HTML e CSS, può essere modificato durante la vita della pagina; la sua vita termina alla chiusura o al ricaricamento. Nell'ambiente-browser è disponibile come oggetto **document**.

6.2 Interfacce e la DOM API

La parola *interfaccia* denota il “fronte esterno” di un servizio, che permette ad altri di utilizzarlo:

- la **UI** (user interface) è il fronte verso gli *utenti*;
- l'**API** (application programming interface) è il fronte verso *altri software / i programmatori*: dice come usare i servizi nel codice;
- l'**object interface** di un oggetto è costituita dalle sue proprietà e dalle *signature* dei metodi: come l'oggetto può essere usato, a prescindere da come implementa il comportamento. È un'astrazione del concetto di classe (la classe contiene anche la componente *operazionale*: come si costruiscono gli oggetti e come i metodi implementano il comportamento); una stessa object interface può essere implementata da classi diverse.

La specifica di un'interfaccia può essere *a posteriori* (realizzo il servizio, poi ne pubblico l'API) o *a priori*, come **contratto**: l'elenco di funzionalità è un requisito che può essere implementato da fornitori diversi (analogamente, un mockup è una specifica a priori di una UI).

Definizione – DOM API

La **DOM API** è un'interfaccia del secondo tipo: una **specifica, stabilita dal W3C**, di come deve essere il servizio di interrogazione e aggiornamento del DOM. Poiché il servizio è offerto ai programmatori JavaScript *dai browser*, sono i browser a giocare il ruolo di fornitori e a doversi attenere alla specifica. È fatta di *oggetti* che devono essere forniti (il DOM stesso e i suoi elementi) e di *object interfaces* che tali oggetti devono rispettare; essendo “solo” una specifica **non contiene componenti imperative, quindi non definisce classi** (sta a chi implementa decidere se usarle).

JavaScript non prevede le object interface come costrutti del linguaggio (servirebbero solo al controllo statico dei tipi, che JS non ha): dire che i nodi DIV “implementano l'interfaccia `HTMLDivElement`” significa solo che quegli oggetti hanno tutte le proprietà e i metodi elencati nella specifica W3C. La gerarchia delle interfacce comprende, fra le altre: `EventTarget` (radice), `Node`, `Window`, `Document`, `DocumentFragment`, `Element`, `HTMLElement` e le sue specializzazioni (`HTMLDivElement`, `HTMLInputElement`, `HTMLParagraphElement`, `HTMLFormElement`, `HTMLImageElement...`), `Attr`, `CharacterData` con `Text` e `Comment`.

Da ricordare – Le tre macrocategorie della DOM API

1. **Navigazione / attraversamento del DOM:** trovare uno o più elementi in base alla struttura dell'albero e/o a tipo, classe, id, e leggerne le caratteristiche. Gestita per lo più da proprietà e metodi dell'interfaccia **Element**.
2. **Modifica del DOM:** aggiungere, rimuovere e spostare elementi; modificare caratteristiche (classi, attributi), contenuti e stili. Anch'essa per lo più su **Element**.
3. **Gestione degli eventi:** registrare (subscribe/unsubscribe) funzioni come *listener* degli eventi UI di interesse, specifiche per tipologia di evento e per elemento sorgente; al verificarsi dell'evento il listener riceve le informazioni dettagliate. Gestita tramite l'interfaccia **EventTarget**.

Lo schema tipico della programmazione UI in “vanilla JavaScript” è un loop su: (1) trovare nel DOM gli elementi su cui intervenire, (2) registrarvi listener, (3) al verificarsi dell'evento, modificare opportunamente il DOM.

6.3 Navigare il DOM

6.3.1 Ricerca per tipo, classe, id

Definiti sia da **Element** (ricerca nel sottoalbero) sia da **Document** (ricerca sull'intero documento):

- `getElementById(id)` → **Element**: l'elemento con quell'id, o `null`;
- `getElementsByName(tag)` → **HTMLCollection** (*live*);
- `getElementsByClassName(cls)` → **HTMLCollection** (*live*).

6.3.2 Attraversamento dell'albero

- da **Document**: `document.documentElement` (la radice), `document.head`, `document.body`;
- da **Element**: `element.children` → **HTMLCollection** (*live*) dei figli che sono a loro volta elementi;
- da **Node**: `node.childNodes` → **NodeList** (*live*) di *tutti* i figli (elementi o meno); `node.parentElement` (il genitore *se è un elemento*, altrimenti `null`); `node.parentNode` (il genitore di qualunque tipo, o `null`).

6.3.3 Ricerca per selettore

Usa i selettori CSS già visti. Discendente (da **Document** o da un **Element**):

- `querySelector(sel)` → il *primo* elemento corrispondente, o `null`;
- `querySelectorAll(sel)` → **NodeList** **statica** di tutti i corrispondenti.

Ascendente (solo da un **Element**): `closest(sel)` → il primo elemento corrispondente *risalendo verso la radice*, o `null`.

Attenzione – HTMLCollection e NodeList: live vs statiche

Sono collezioni *array-like* ma **non array**: non supportano i metodi di **Array**, ma permettono `for` classici e `for..of` (introdotte agli albori della DOM API, mantenute per retrocompatibilità). Le ricerche per tag/classe e le proprietà di attraversamento restituiscono collezioni **live**: se il DOM cambia, i contenuti si aggiornano. La ricerca per selettore restituisce collezioni **statiche**: per risultati aggiornati bisogna *ripetere la query*. Conversione: `Array.from(coll)` – ma l'array risultante è sempre e comunque statico.

6.4 Modificare il DOM

6.4.1 Contenuti (modifica bulk)

- `element.innerText` / `element.innerHTML` → string: il contenuto come testo o come HTML; *modificandoli si sostituisce completamente il sottoalbero* che fa capo all'elemento.

6.4.2 Creazione, rimozione, inserimento

- `document.createElement(tag)`: crea un `Element` *non ancora nel DOM* (radice di un sottoalbero a parte);
- `element.remove()`: elimina l'elemento dal DOM;
- i tre metodi “ortodossi” di inserimento, in posizione contigua all'elemento su cui sono chiamati:
 - `insertAdjacentElement(where, anElem)`: `anElem` può essere nuovo (da `createElement`) oppure già nel DOM – in tal caso viene **spostato** (con tutto il suo sottoalbero);
 - `insertAdjacentHTML(where, htmlString)`: costruisce un nuovo sottoalbero dalla stringa HTML e ne inserisce la radice;
 - `insertAdjacentText(where, textString)`: crea e inserisce un nodo `Text`.

La posizione `where` è una stringa fra: **beforebegin** (fratello *predecessore*: subito prima del tag di apertura), **afterbegin** (*primo figlio*: subito dopo il tag di apertura), **beforeend** (*ultimo figlio*: subito prima del tag di chiusura), **afterend** (fratello *successore*: subito dopo il tag di chiusura).

6.4.3 Classi e attributi

- `element.classList.add(c)` / `.remove(c)` / `.toggle(c)`: aggiungono/tolgono/alternano una classe fra quelle dell'attributo `class`; `.has(c)` dice se l'elemento la possiede. *Poiché cambiando le classi cambiano le regole di stile applicate, è un modo molto efficace di ottenere interattività*: far apparire/scompare elementi, evidenziarli, spostarli.
- `element.getAttribute(nome)`: valore dell'attributo (come stringa);
- `element.setAttribute(nome, valore)`: assegna il valore all'attributo.

Nota: modificare il DOM (smontare/rimontare un elemento) è cosa diversa dal semplice `display:none`.

6.5 L'event loop asincrono single-thread

L'ambiente di esecuzione JavaScript interagisce col mondo esterno tramite **eventi**, che possono essere:

- **esogeni**: generati spontaneamente da un agente esterno (tipicamente l'utente); completamente imprevedibili – non è dato dire se e quando avverranno;
- **endogeni**: conseguenza di una *richiesta al mondo esterno* generata dall'applicazione stessa (es. una `fetch`); è ragionevolmente certo che si verificheranno, salvo errori.

Gli eventi **si mettono in coda**; perché vengano gestiti, l'ambiente deve conoscere la funzione che risponde all'evento:

- **listener** per gli eventi esogeni: sta “in ascolto” del verificarsi dell'evento (non prevedibile) e risponde ogni volta che accade;
- **callback** per gli endogeni: a ogni avvio della richiesta si specifica la funzione da richiamare (call back) al completamento.

La logica è la stessa: istruzioni che *associano a un tipo di evento una funzione* da eseguire quando l'evento si verifica.

Da ricordare – Single-thread e non bloccante

JavaScript “consuma” la coda degli eventi con un **singolo thread**: gli eventi sono gestiti in sequenza, nell'ordine di arrivo. Se la gestione di un evento intraprende un'elaborazione lunga, *blocca l'intera applicazione*: per questo le elaborazioni time-consuming (richieste HTTP, scrittura nel DB del browser) sono gestite **in modo asincrono** – le istruzioni che le avviano non si bloccano in attesa del risultato, ma richiedono una callback che verrà invocata a calcolo terminato. (È in realtà possibile eseguire compiti specifici, che non toccano il DOM, in thread paralleli detti **Web Workers**, ma il core dell'esecuzione resta come descritto.)

6.6 Gli eventi del DOM

Gli oggetti del DOM ricevono eventi in corrispondenza delle interazioni supportate:

- tutti gli elementi: dispositivi di puntamento – `click`, `mousedown`, `mouseup`, `mouseenter`, `mouseleave`, `mouseover`, `mouseout`, `mousemove` (sui touch screen gli eventi touch si possono trattare a parte o assimilare al mouse);
- molti elementi di input: tastiera – `focus`, `blur`, `keydown`, `keyup`;
- gli elementi form: interazioni di livello più alto – `submit`, `reset`;
- `document`: `scroll`; `window`: `resize`.

L'elenco esaustivo è lunghissimo, ma per la maggior parte delle applicazioni bastano gli eventi del mouse.

6.6.1 Registrare un listener

Tutti gli oggetti del DOM che ricevono eventi implementano:

```
domObj.addEventListener(nomeEvento, listener);  
domObj.removeEventListener(nomeEvento, listener);
```

`nomeEvento` è una stringa ("`click`", "`mouseup`", "`keydown`"...); il `listener` è una funzione con un parametro opzionale e nessun valore di ritorno, chiamata quando l'evento si verifica su `domObj`; riceve un *oggetto evento* con le informazioni dettagliate. La DOM API definisce le interfacce di questi oggetti: `Event` e le sue specializzazioni `MouseEvent`, `KeyboardEvent`, `WindowEvent`, `FocusEvent`, eventi form... Gli eventi UI sono per natura *asincroni* e vanno gestiti tenendone conto.

6.7 Bubbling e capturing

Definizione – Bubbling

Gli eventi del DOM si generano su un elemento preciso e da lì **risalgono l'albero fino alla radice**: l'evento viene emesso sia dall'elemento sorgente sia da tutti gli elementi sul percorso verso la radice. Nel listener, dall'oggetto evento si distinguono:

- `evento.target`: l'elemento *sorgente*;
- `evento.currentTarget`: l'elemento che sta emettendo l'evento *in quel momento* (presumibilmente quello a cui abbiamo attaccato il listener).

Perché il bubbling? Se volessimo sapere quando l’utente clicca “sul footer”, senza bubbling riceveremmo il click solo sui punti del footer *senza* elementi interni (lo sfondo) – raramente ciò che serve: “click sul footer” di solito significa “footer incluso ciò che contiene”. Il bubbling mette in pratica questo concetto. Per **bloccare** la propagazione oltre un certo elemento (esempio tipico: il *glasspane* delle finestre modali), nel listener si usa `evento.stopPropagation()`;

Definizione – Capturing

La specifica stabilisce che, prima del bubbling, l’evento *giunga* al target **discendendo l’albero**: questa fase è il **capturing**. Di default è nascosta (durante la discesa i listener non vengono invocati); si può chiedere l’invocazione in fase di capturing con:

```
domObj.addEventListener(nomeEvento, listener, {capture: true});
```

Anche qui è possibile `stopPropagation()`: bloccando in capturing, l’evento *non arriva mai* al target e il bubbling non inizia – utile per gestioni totalmente customizzate di certi eventi.

6.7.1 Effetti di default

Alcuni eventi hanno effetti predefiniti: il click su un elemento A naviga altrove, il tasto destro apre il menu contestuale, TAB passa al controllo successivo... Si impediscono nel listener con `evento.preventDefault()`; . Per bloccarli in tutta un’area dell’applicazione conviene farlo *in fase di capturing*.

7. TypeScript e la gestione delle dipendenze

7.1 Carta d'identità di TypeScript

Definizione – TypeScript

TypeScript aumenta la sintassi di JavaScript con un **sistema di tipi**:

- si può dichiarare il *tipo atteso* di variabili, parametri e valori di ritorno delle funzioni;
- è un linguaggio **compilato** (traspilato): i programmi TS vengono tradotti in programmi JS. Di conseguenza i costrutti aggiuntivi di TS sono **esclusivamente dichiarativi e non operazionali**: “scompaiono” con la compilazione e non hanno alcun impatto sull'esecuzione;
- implementa un sistema di **inferenza dei tipi**: a compile-time determina il tipo risultante di funzioni ed espressioni e verifica la consistenza con i tipi “promessi” o con l'uso dei risultati;
- dà il meglio di sé **integrato in un IDE**: l'inferenza viene applicata già mentre si scrive, prevenendo molti errori prima ancora della compilazione.

Da ricordare – Tipi statici vs dinamici

In linguaggi come JavaScript o C il tipo influenza lo *slot di memoria* usato per il dato; in TypeScript no: le dichiarazioni di tipo **non hanno effetto a run-time**, perché svaniscono nella compilazione. A cosa servono allora? A permettere, *a compile-time*, la verifica della consistenza dei tipi in espressioni e assegnamenti: il programmatore esprime le proprie *intenzioni* e il compilatore (e l'IDE) rileva le discrepanze fra intenzione e codice scritto. Permettono inoltre di controllare che proprietà e metodi a cui si accede *esistano davvero*, e di introdurre nelle classi i *modificatori di accesso* (public, private...) assenti in JavaScript.

Attenzione – Il tipo any

Si può “non esprimere” il tipo di un'entità dichiarandolo **any** (qualsiasi cosa), ma così si perdono buona parte dei vantaggi di TypeScript: da evitare, salvo i rari casi in cui si deve far convivere codice TS con codice JavaScript privo di dichiarazioni.

7.2 Package management: npm

Con TypeScript il progetto inizia a dipendere dal compilatore **tsc** e dai *package* che definiscono il linguaggio. È un'istanza di un caso generale: nello sviluppo web è comune usare package di terze parti, distribuiti tramite *repository online*. Domande tipiche: quando ridistribuire il progetto perché lo richiedono le dipendenze? Come evitare che l'aggiornamento di un package “rompa” applicazioni già distribuite? Che succede se uno sviluppatore ritira il proprio package?

Definizione – Package manager

Un **package manager** è un tool (tipicamente da linea di comando) che permette di accedere al registry/repository abbinato, scaricare e installare i package necessari e *dichiararne la dipendenza* tramite opportuni dependency file. Il più diffuso nel mondo JS/TS è **npm** (node package manager), col registry **npmjs.com**; il modo più semplice di installarlo è installare

Node.js, che lo include.

```
> npm install [-D] [-g] nome-package[@versione]
```

- **-D**: *dev dependency* (default: runtime dependency);
- **-g**: installazione *globale* (default: locale al progetto);
- **@versione**: quale versione installare; **@latest** = ultima stabile, **@next** = prossima uscita (non stabile).

Da ricordare – Dipendenze: locali vs globali, dev vs runtime

Locali vs globali: npm può installare i package a livello globale (come librerie di sistema) o *locale*, nella cartella del progetto. La strategia più comune oggi è l'installazione **locale**: rende lo sviluppo indipendente dalla configurazione del dispositivo e facilita la collaborazione via Git/GitLab. **Dev vs runtime**:

- *dev dependencies*: servono in fase di sviluppo ma “scompaiono” alla distribuzione – TypeScript è una di queste (compilato in JS, compilatore e librerie TS non servono più);
- *runtime dependencies*: codice effettivamente invocato durante l'esecuzione, da includere nella distribuzione.

La distinzione è rilevante nel “confezionamento” dei progetti: sviluppo leggibile, distribuzione concisa ed efficiente.

7.2.1 Struttura di progetto npm

- **node_modules**: il codice dei package; gestita interamente da npm, non va modificata e **va esclusa da Git**;
- **package.json**: configurazione del progetto e dipendenze; npm ci scrive, ma è modificabile anche a mano (es. "devDependencies": {"typescript": "^5.6.3"});
- **package-lock.json**: dipendenze e sotto-dipendenze; generato da npm, da non modificare.

7.3 Configurare e compilare TypeScript

Il compilatore **tsc** è uno script JavaScript contenuto nel package typescript, eseguito tramite il tool **npm** (incluso in npm).

```
> npm install -g typescript # crea tsconfig.json con impostazioni predefinite
> npm run tsc               # compila: cerca i .ts e salva i .js nella outDir
```

Le **compilerOptions** principali in **tsconfig.json**:

- **target**: versione di JavaScript dell'output. Default **es2016** (eseguibile da tutti i browser); nel corso si usa **es2020** per avere più feature (ormai ampiamente supportato);
- **module**: come gestire i moduli. Prima che ECMAScript introducesse import/export, Node.js aveva una propria sintassi (“CommonJS”); poiché scriviamo per il browser e vogliamo la sintassi ES, si indica **es2015** (la versione che ha introdotto import/export); per Node moderno si userebbe **nodenext**;
- **esModuleInterop**: interoperabilità fra le due modalità di specifica dei moduli (rilevante soprattutto programmando per Node con stile ES);
- **forceConsistentCasingInFileNames**: il compilatore segnala import di uno stesso modulo con uso inconsistente di maiuscole/minuscole (problema fra S.O. che distinguono e S.O. che no);

- **strict**: maggiore severità nella verifica dei tipi;
- **skipLibCheck**: non verifica i tipi dentro le librerie;
- **outDir**: *dove* mettere i file compilati. Conviene tenere i sorgenti **.ts** in una cartella **src** e compilare in **scripts**: le pagine HTML si riferiscono a **scripts** (la “vera” applicazione da distribuire) e ignorano i sorgenti; viceversa ai collaboratori serve solo **src**, perché **scripts** si rigenera compilando. tsc replica in output la struttura di cartelle dei sorgenti.

7.4 Dichiarazioni di tipo

```
let nomevar: TIPO;
const c: TIPO = espressione;
function fun(p: TIPO, q: TIPO): TIPORET {...}
const arr = (a: TIPO, b: TIPO): TIPORET => ...
```

Le dichiarazioni di tipo di *variabili e parametri* sono obbligatorie, a meno che non venga subito assegnato un valore (TS inferisce il tipo dall’espressione). Il tipo di *ritorno* non è obbligatorio (inferito dalle **return**), ma dichiararlo esplicitamente è utile: esprime le intenzioni, e se il valore restituito è del tipo sbagliato – o manca la **return** – il compilatore lo segnala.

7.5 Espressioni di tipo

Imparare TypeScript significa soprattutto saper scrivere le **espressioni di tipo** – meno banale di quanto sembri (“una funzione che prende un array di interi e restituisce una stringa”...). Scriverle correttamente è una forma preliminare di *testing*: se il codice supera i controlli di tipo, molti errori grossolani sono già esclusi.

7.5.1 Tipi base, array, tuple

- nomi dei tipi base: **string**, **boolean**, **number**, **undefined**, **null**, **bigint**, **symbol**;
- il nome di una **classe**: la variabile conterrà solo oggetti di quella classe;
- **void**: tipo di ritorno di una funzione che non ritorna nulla;
- **array**: tipo degli elementi + quadre – **let mioArray: number[]; let iscritti: Studente[];**
- **tupla**: array di lunghezza fissa con tipi posizionali noti – **let tripla: [number, string, Studente].**

7.5.2 Tipo unione

Composizione con **|**: un valore “di tipo A oppure B”.

```
let misto: (number | string)[]
function trovaStudente(matricola: number): Studente | null {...}
// unione di tipi literal (elenco di valori possibili):
let cifra: 0|1|2|3|4|5|6|7|8|9;
let primavera: "marzo" | "aprile" | "maggio";
```

7.5.3 Tipo funzione

Si costruisce da tipi dei parametri e del ritorno: **(p1: T1, ..., pN: TN) => TRET**. I “nomi” **p1...pN** sono solo segnaposto.

Esempio

`superCalcolo` lancia un calcolo remoto e prende una callback che riceverà il risultato (array di interi) restituendo un boolean:

```
function superCalcolo(datiInput: number[],
    callback: (datiOutput: number[]) => boolean): void {...}
// e per memorizzare superCalcolo in una variabile:
let somevar: (x: number[], y: (z: number[]) => boolean) => void;
somevar = superCalcolo;
```

7.5.4 Le classi in TypeScript

Una classe TS *dichiara le proprietà* (variabili di istanza) degli oggetti, con i tipi:

```
class Studente {
    nome: string;
    matricola: number;
    votiEsami: number[];
    ...
}
```

Costruttore e metodi come in JS, ma con i tipi dichiarati. Inoltre:

- **visibilità:** `private`, `public` (default), `protected`; in più `readonly` (assegnata nel costruttore e mai più modificata);
- TS verifica che tutte le variabili d'istanza siano *inizializzate nel costruttore*; per saltare il controllo (perché verrà inizializzata dopo, in conseguenza di un evento) si pospone `!`; per dichiararla *opzionale* (potrebbe non avere valore) si pospone `?`;
- proprietà e metodi devono *sempre rispettare la dichiarazione della classe*: le uniche proprietà aggiungibili/rimovibili durante la vita dell'oggetto sono quelle opzionali (`?`).

7.5.5 Le interface**Definizione – Interface**

Le **interface** sono un costrutto *completamente dichiarativo*, senza semantica operativa: non se ne trova traccia nel codice compilato. Servono a garantire che gli oggetti in input/output abbiano determinate proprietà – importante in un linguaggio dove si creano spesso oggetti *senza costruttori*. Un'interface è un elenco di proprietà (anche proprietà-funzione); il suo nome definisce un *tipo*; con `?` si dichiarano proprietà opzionali.

```
interface Book {
    title: string;
    author: string;
    year: number
}
```

Quando un oggetto *rispetta* un'interfaccia I?

- un **object literal** la rispetta se *corrisponde esattamente*: tutte le proprietà non opzionali, e *nessuna* proprietà aggiuntiva;

- un oggetto di una **classe** **C** la rispetta se **C** dichiara tutte le proprietà non opzionali di **I**; **C** può averne anche altre. Una classe che definisce tutte le proprietà di **I** si dice che **implementa** l'interfaccia, esplicitabile con `class C implements I {...}`.

La versione TypeScript della DOM API definisce come interface tutti i tipi di oggetti che usa. Molte librerie nate in JavaScript forniscono definizioni di tipo parallele in file `.d.ts` (es. i tipi della DOM API sono in `lib.dom.d.ts`, nel package `typescript`).

7.5.6 Dizionari indicizzati

In JS/TS è comune usare gli oggetti come “dizionari” (simili a hash map) per reperire valori da una chiave. Ma TypeScript *non permette* di accedere a proprietà non menzionate dalla definizione di tipo – salvo che l'oggetto sia dichiarato esplicitamente come dizionario indicizzato:

```
const repo: {[chiave: string]: Persona} = {};
function memorizza(p: Persona) { repo[p.codiceFiscale] = p; }
```

Le chiavi possono essere soltanto `string` o `number`; i valori di qualsiasi tipo.

7.5.7 Accessors: getter e setter

Caratteristica di JavaScript (non di TypeScript), ma la sua utilità emerge con i tipi. Permettono di definire **proprietà “calcolate”**: utile ad esempio quando un'interfaccia richiede *proprietà* che per una classe sarebbero naturalmente *metodi* (calcolabili da altre proprietà).

```
interface RagioneSociale { denominazione: string; codiceFiscale: string; }
class Persona {
  get denominazione(): string { return `${this.nome} ${this.cognome}`; }
  set denominazione(den: string) {
    const sep = den.split(" ");
    this.nome = sep[0]; this.cognome = sep[1];
  }
}
tizio.denominazione = "Mickey Mouse"; // invoca il setter
const s = tizio.denominazione;        // invoca il getter
```

Regole:

- il `get` non prende parametri e restituisce il valore; il `set` non restituisce nulla e prende il nuovo valore;
- se presenti entrambi, il tipo di ritorno del `get` deve corrispondere al parametro del `set`;
- non è obbligatorio averli entrambi: solo `get` $\Rightarrow$ proprietà *readonly*; solo `set` $\Rightarrow$ scrivibile ma non leggibile.

Usi tipici: proprietà calcolate da altri valori; controllo degli accessi senza rendere private le proprietà interne (limitare valori numerici, validare formati, elaborare prima di restituire).

7.5.8 Generics

Definizione – Generics

I **generics** esprimono un comportamento *indipendente dal tipo di dato* su cui operano: un ordinamento funziona su array di tipi diversi (restituendo un array dello stesso tipo); uno Stack ha push/pop analoghi qualunque sia il tipo degli elementi. TypeScript permette funzioni, classi e interfacce *generiche*, senza doverle ridefinire per ogni tipo. La sintassi usa le parentesi

angolari; con una *variabile di tipo* si creano elementi generici:

```
const insiemeNum = new Set<number>()
const insiemeStudenti = new Set<Studente>()
function middle<T>(lista: T[]): T {
  return lista[lista.length/2];
}
```

(Se una classe è generica, lo è anche il suo costruttore.)

8. Richieste HTTP dal client: Fetch, Promise e JSON

8.1 La fetch API

Definizione – fetch

Nell'ambiente browser, per effettuare richieste HTTP verso un server si usa la **fetch API**, che consta in sostanza di un'unica funzione:

```
fetch(url: string, init?: RequestInit): Promise<Response>

interface RequestInit {
  method?: string;
  headers?: { [key: string]: string };
  body?: string | null;
}
```

- **url**: la risorsa richiesta. Può essere *esterna* (con dominio), *assoluta* o *relativa*; per assolute/relative il dominio è quello di provenienza; per le assolute il punto di partenza è la radice del dominio, per le relative la location della pagina HTML in cui gira lo script.
- **init** (opzionale): oggetto che rispetta **RequestInit**. Proprietà più usate: **method** (default "GET"), **headers** (dizionario chiave/valore con gli header HTTP), **body** (per i method che inviano dati, POST su tutti; per i nostri scopi sempre una string). Per una GET spesso si omette del tutto: i default vanno bene.

8.1.1 AJAX e la storia della fetch

La fetch API sostituisce la precedente **XMLHttpRequest**. La “novità” (dal 2015) è che ingloba il paradigma **AJAX**: prima di AJAX, la risposta a una richiesta HTTP fatta da JavaScript veniva intercettata dal *browser*, che la usava per visualizzare una nuova pagina (resettando l'ambiente di esecuzione). Con AJAX la risposta viene ricevuta *dal programma stesso* tramite una callback: il programma può usare i dati per modificare il DOM **senza ricaricare la pagina**. Prima della fetch servivano librerie apposite per AJAX; la fetch lo include: una richiesta fatta con **fetch** torna sempre al programma che l'ha generata. **Axios** è una di quelle librerie pre-fetch, ancora usata per due ragioni: richieste un po' più semplici da formulare, e disponibilità anche *fuori* dall'ambiente browser (la fetch API è solo nel browser).

8.1.2 Perché fetch non prende una callback?

fetch è **asincrona**: avvia la trasmissione e restituisce subito il controllo; l'arrivo della risposta (o del timeout) è un *evento endogeno*. Il modo “classico” sarebbe una callback fra i parametri – ma nella stessa edizione di JavaScript è stata introdotta la **Promise API**: un meccanismo più sofisticato e meno suscettibile di errori per gestire gli eventi endogeni. Una operazione asincrona non restituisce un dato, ma *la promessa di un dato che arriverà*. La Promise API è generica (per qualsiasi operazione asincrona); la fetch la sfrutta.

8.2 I problemi del meccanismo di callback

Esempio: una funzione `update` che (1) richiede una risorsa web, (2) all'arrivo la stampa e legge dati voluminosi da un DB, (3) all'arrivo dei dati aggiorna la visualizzazione. Due operazioni asincrone, ognuna con la sua callback; la seconda va avviata *dentro* la callback della prima. Problemi:

- **il codice è scritto “a rovescio”** rispetto alle tempistiche reali: le callback vanno definite prima di essere invocate, ma l'ordine di esecuzione è dato dalle invocazioni, non dalle definizioni;
- se il resto del programma deve sapere quando `update` termina, anche `update` deve richiedere una callback... e far sì che riceva “a cascata” i risultati delle elaborazioni precedenti diventa rapidamente complicato;
- ad ogni livello di indirezione è più difficile ricostruire la sequenza reale delle operazioni (persino quando sono del tutto sequenziali): il codice si *disperde*, e in esecuzione non si sa più chi aveva chiamato la callback – le conseguenze di un errore diventano quasi impossibili da tracciare all'indietro.

8.3 Le Promise

Definizione – Promise

Una **Promise** è un oggetto che rappresenta *l'esito* (ancora da venire, o già completato) di un'operazione asincrona. Con le Promise, una funzione asincrona restituisce subito il controllo *restituendo un oggetto* `Promise<T>` (è un generic: `T` è il tipo del valore atteso al completamento). Il chiamante non delega la gestione del risultato alla funzione asincrona (come con le callback), ma *aggancia* il proprio codice all'oggetto Promise. La sostanza non cambia: le Promise non rendono possibile nulla che prima non lo fosse – rendono il codice più leggibile e fanno sì che l'ordine di scrittura sia il più possibile affine all'ordine di esecuzione.

Da ricordare – I tre stati di una Promise

- **Pending:** operazione in corso; la Promise non è risolta, il risultato non è disponibile. *Tutte le Promise nascono pending.*
- **Fulfilled:** operazione terminata con successo; la promessa è mantenuta, il risultato è disponibile.
- **Rejected:** operazione fallita; la promessa non può essere mantenuta, il risultato non sarà mai disponibile; può essere disponibile un oggetto con le informazioni sull'errore.

8.3.1 then, catch, finally

A una Promise si *aggancia* una funzione che viene eseguita quando lo stato passa da pending a fulfilled o rejected; il *modo* in cui la si aggancia decide quando viene eseguita:

- `p.then(fun)`: `fun` chiamata in caso di **successo**, col dato ottenuto;
- `p.catch(fun)`: chiamata in caso di **fallimento**, con l'oggetto errore;
- `p.finally(fun)`: chiamata **in ogni caso**, all'uscita da pending.

Tutti e tre **restituiscono una nuova Promise**, il cui esito dipende dalla Promise originale e da ciò che accade nella funzione agganciata; l'oggetto “promesso” è quello restituito dalla funzione agganciata. Si creano così *catene* di funzioni chiamate in successione; la Promise finale può essere restituita al chiamante, che vi attaccherà le proprie funzioni.

then in dettaglio. Data `x: Promise<T>` e `y = x.then(fun)`:

- **fun** può ricevere un parametro di tipo **T** (o nessuno); viene chiamata solo se e quando **x** diventa fulfilled;
- **y** è una **Promise<U>**, dove **U** è il tipo restituito da **fun** (eventualmente **void**); *caso particolare*: se **fun** restituisce una **Promise<X>**, allora **y** è a sua volta **Promise<X>** (“una promessa di una promessa è una promessa”);
- se **x** ha successo, anche **y** diventa fulfilled (ma se **fun** restituisce una terza Promise **z**, **y** segue successo/fallimento di **z**);
- se **x** fallisce con errore **err**, anche **y** diventa rejected con lo stesso **err**.

catch in dettaglio. **u = x.catch(fun):** **fun** riceve l’errore (o nulla) e viene chiamata solo al passaggio a rejected; **u** segue le stesse regole di tipo del **then**. Se l’errore viene correttamente gestito, **u** può diventare fulfilled; se **fun** lancia (throw) un errore **err2**, **u** diventa rejected con **err2**. Di solito *catch si concatena dopo then*.

finally in dettaglio. **w = x.finally(fun):** **fun** viene chiamata all’uscita dallo stato pending, con qualunque esito; **w** è una **Promise<T>** che si comporta come **x** (il risultato di **fun** viene ignorato), salvo che se **fun** lancia un errore o restituisce una Promise che fallisce, **w** fallisce. Anche **finally** permette di proseguire il concatenamento.

8.3.2 Combinare più operazioni asincrone

Date **N** operazioni asincrone **op1...opN**:

- **Congiunzione** – conclusa con successo solo se e quando *tutte* le **opI** terminano con successo:

```
const conj: Promise<T1,T2,T3> = Promise.all([p1, p2, p3])
```

fulfilled quando **p1**, **p2**, **p3** sono tutte fulfilled, con l’array dei risultati; rejected *non appena una* fallisce, con l’errore della prima fallita.

- **Disgiunzione** – conclusa con successo se e quando *una* termina con successo:

```
const disj: Promise<T1|T2|T3> = Promise.any([p1, p2, p3])
```

fulfilled quando la *prima* è fulfilled, col suo risultato; rejected se e quando *tutte* falliscono, con un **AggregateError** che raccoglie gli errori.

- **Competizione** (variante della disgiunzione) – successo solo se una termina con successo *prima che un’altra fallisca*:

```
const comp: Promise<T1|T2|T3> = Promise.race([p1, p2, p3])
```

si risolve quando la *prima* si risolve, col medesimo esito della “vincente” (fulfilled se vince una fulfilled, rejected se vince una rejected).

- **Concatenazione** – **op_i** inizia solo se e quando **op_{i-1}** ha avuto successo; successo complessivo = successo di **opN**. Si realizza con catene di **then**:

```
const chain: Promise<T3> = op1(...)
  .then((d: T1) => op2(...))
  .then((d: T2) => op3(...));
```

8.3.3 async / await

Definizione – async/await

Parole chiave che *semplificano scrittura e leggibilità* delle concatenazioni di Promise (non aggiungono funzionalità: il codice è traducibile in catene di then).

- **async** si applica alla definizione di una funzione/metodo che restituisce una `Promise<T>`: abilita **await** nel corpo; al termine si può restituire un `T`, incapsulato implicitamente in una Promise.
- **await** si premette all'invocazione di una funzione che restituisce `Promise<U>`: permette di trattarla come una normale funzione che restituisce un `U`; è implicito che il codice successivo verrà eseguito solo alla risoluzione della Promise.

```
async function someFunction(): Promise<T3> {  
  const x1: T1 = await op1(...);  
  const x2: T2 = await op2(...);  
  const x3: T3 = await op3(...);  
  return x3;  
}
```

8.4 Il formato JSON

Definizione – JSON

Il formato **JSON** (JavaScript Object Notation) nasce in ambiente JavaScript per la **serializzazione** di oggetti, principalmente ai fini della trasmissione in una connessione HTTP. *Serializzare* = rappresentare oggetti come stringhe (per salvarli su file o inviarli come body HTTP); l'operazione deve essere *invertibile* (*deserializzazione*: ricostruire l'oggetto dalla stringa). Una stringa JSON è costituita da un singolo *valore JSON*, che può essere:

- un **valore semplice**: number, string, boolean, null;
- un **array JSON**: valori separati da virgole fra quadre [];
- un **oggetto JSON**: coppie "chiave": valore separate da virgole fra graffe { }, dove la chiave è una *stringa fra virgolette*.

Differenze dalla sintassi degli oggetti JavaScript: valori semplici più limitati (niente bigint, symbol, undefined); *non si possono esprimere funzioni*; le chiavi vanno fra virgolette. Con l'evoluzione del web, JSON si è esteso alla rappresentazione di informazioni strutturate in generale (tsconfig.json, package.json) ed è oggi un formato di interscambio **language-independent**: anche i linguaggi server-side (Java, PHP, Python...) hanno librerie per serializzare/deserializzare JSON (il mondo Java tende storicamente a preferire XML per gli stessi scopi).

8.4.1 Serializzazione e deserializzazione

- `JSON.stringify(ogg)`: da oggetto/array/valore a stringa JSON. Esplora l'oggetto *ricorsivamente, in profondità*. Può:
 - *ignorare* valori non serializzabili (undefined e funzioni non vengono serializzati);
 - *dare errore* se la struttura non è serializzabile, in particolare con **riferimenti circolari** (es. albero in cui il padre riferisce i figli e i figli il padre: loop infinito della serializzazione).

Quindi non tutti gli oggetti sono serializzabili, o la serializzazione può non preservare tutte le informazioni. Applicata a oggetti di una classe, **l'appartenenza alla classe si perde**: l'oggetto diventa a tutti gli effetti un object literal.

- `JSON.parse(str)`: da stringa a object literal / array / valore semplice. L'oggetto *non* può essere “inserito d'ufficio” in una classe (solo il costruttore crea oggetti di una classe); ma se sappiamo che rispetta un'interface possiamo tiparlo: `const libro = JSON.parse(stringaDati) as Book`.

8.4.2 JSON nelle richieste e risposte HTTP

POST con body JSON dall'app client-side:

```
fetch(someUrl, {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify(objectToSend)
})
```

Leggere il body di una Response:

```
fetch(...)
  .then((res: Response) => res.json())
  .then((dati: MyExpectedDataType) => { /* usa i dati */ });
```

Attenzione

Il body di una **Response non è una string** ma un **ReadableStream**: all'arrivo della risposta i dati potrebbero non essere ancora interamente pervenuti (si pensi a un video o un'immagine). Anche aspettando una JSON string che arriva in tempi brevi, bisogna chiamare `res.json()`, che restituisce una *Promise* dell'oggetto contenuto nel body: mette in attesa che i dati siano interamente trasferiti e quindi deserializzati.

8.5 La server-side API

Una server-side app (per i nostri fini: risponde a richieste HTTP scambiando principalmente JSON) espone un accesso al **Data Model**, gestito più naturalmente lato server perché è il server a poter rendere persistenti i dati (con un database); l'app server-side è la *source of truth* del data model. (La persistenza lato client è possibile ma limitata al singolo utente su un singolo dispositivo.) App più complesse costruiscono sopra i dati una *business logic* più articolata, con servizi “live” che combinano i dati del DB con informazioni circostanziali o di terze parti.

L'app client *usa* le funzionalità dell'app server; questa deve dichiarare la propria **API**, specificando per ogni funzionalità: come si invoca, quali input richiede, quali sono gli effetti e gli output. Su HTTP:

- **Qualificare la funzionalità**: tramite il **pathname** della URL (ogni funzionalità su un pathname diverso; la sequenza di segmenti “instrada” – si parla di **route**) e il **method** (più funzionalità sullo stesso pathname purché con method diverso). In linea di massima: **GET** per funzionalità che forniscono dati dal server al client, **POST** per quelle che inseriscono nuovi dati nel Data Model (es. `GET /bookshelf/collections` ottiene l'elenco delle raccolte, `POST /bookshelf/collections` ne aggiunge una). È una semplificazione: nei casi che non vi rientrano si sceglie in base all'input richiesto.
- **Esprimere gli input**, con tre aspetti della richiesta:

- il **body** (solo POST, restringendoci a GET/POST): per input “corposi” e strutturati (array, oggetti);
- la **query string** (GET o POST): ogni coppia chiave/valore è un input; più “elegante” per valori semplici o poco strutturati (anche se in teoria qualsiasi cosa è codificabile come stringa);
- la **route parametrica**: alcuni segmenti del pathname sono variabili e corrispondono a un input – es. GET `/bookshelf/books/:id`, invocata come `/bookshelf/books/12`. Tipica per input che *identificano una risorsa* (id, nomi: la URL è un Resource Locator!); funziona per numeri o brevi stringhe.
- **Dichiarare effetti e risposta** (tramite documentazione): le modifiche al Data Model in dipendenza degli input, e come sono fatti i risultati inviati nel body della risposta.

9. Applicazioni client-side con React

9.1 Architetture web: i flussi di informazione

Elementi principali di un'architettura web: **Data Persistence**, **Application Business Logic**, **User Interface**, più eventuali **Third Party Services**. I flussi tipici:

- **Flow 1 – retrieval + presentation**: dai dati persistenti, attraverso la BL, alla UI (la BL elabora i dati ma può elaborare anche elementi di *presentazione*);
- **Flow 2a – user action, UI-only**: l'azione tocca solo la UI (spostare un elemento, selezionare);
- **Flow 2b – user action, non-persistent**: modifica ai dati non ancora da salvare, o azione che influenza la presentazione ma non i dati;
- **Flow 2c – user action con cambiamento persistente**: l'azione arriva fino alla persistenza;
- **Flow 3 – external update**: come il flow 1, ma originato da un *evento esterno* ad applicazione già attiva, non dall'utente.

La logica può essere distribuita: **client-side logic** (BL sul client), **server-side logic** (BL sul server), **two-sided logic** (divisa fra i due lati).

9.2 React: caratteristiche principali

Definizione – React

React è propriamente una *libreria* (non un framework): può essere usato da solo o dentro un framework che lo supporta (“il” framework di React è Next.JS). Nel corso si usa come libreria in abbinamento al tool **Vite** che, con npm, facilita configurazione e gestione del progetto. React supporta lo sviluppo di front-end con **UI orientate ai componenti** (come i competitor Angular e Vue, che però sono framework a tutto tondo) e di **Single Page Application**: applicazioni client-side costituite da un'*unica pagina HTML* il cui DOM viene costantemente aggiornato dall'applicazione stessa. A differenza di altri framework, React **non è object-oriented**: è fortemente basato sugli aspetti *funzionali* di JS/TS.

Libreria vs framework. Una *libreria* è un insieme di package importabili di cui si usano le funzionalità; React è complessa (include un'estensione del linguaggio che richiede compilazione, e un *runtime* di oggetti e funzioni che gestiscono i componenti), ma sono sempre funzionalità usate *dal nostro codice*. Un vero *framework* realizza qualcosa di simile a una “virtual machine”: un ambiente di esecuzione dentro l'ambiente di esecuzione, che carica ed esegue il nostro codice in un contesto altamente modificato, compiendo operazioni “dietro le quinte” (es.: Angular monitora tutti i cambiamenti delle proprietà visualizzate; Vue crea copie “addizionate” degli oggetti). I framework sono più completi e coprono anche aspetti oltre la UI; in React questi aspetti sono demandati a librerie separate.

9.3 Componenti UI e gerarchie

Definizione – Componente UI

Un componente UI è caratterizzato da:

- un **aspetto grafico** (il *template*): come viene visualizzato – sul web client-side, una combinazione di HTML e CSS;
- una **parte del Data Model** visualizzata e/o modificata attraverso il componente (esistono anche componenti che mostrano informazioni statiche o servono solo alla navigazione/interazione);
- un **comportamento**: quali operazioni può compiere l'utente attraverso il componente e come si svolge l'interazione.

Da ricordare – Genitori e figli: sono le istanze a essere figlie

Un componente può essere costruito a partire da (istanze di) altri componenti: è detto **genitore** di quelle istanze, che sono i suoi **figli**. Esempio classico: un menu di navigazione, in cui la singola voce è un componente istanziato più volte (stesso aspetto e comportamento, contenuto diverso), e il menu costruisce altro sul comportamento delle voci (es. garantire che una sola sia selezionata). **Non è il componente in astratto a essere figlio, ma le sue istanze**: istanze diverse dello stesso componente possono avere genitori diversi. La definizione di un componente non include “chi lo conterrà”, ma solo come chi lo conterrà potrà comunicare con esso; un componente in astratto può invece dirsi *genitore*, perché la sua definizione specifica come è costruito.

9.3.1 Componenti dinamici: show/hide vs mount/unmount

I componenti cambiano aspetto in base a dati e interazioni – incluso il “set” di figli mostrati (un menu espandibile: compresso non mostra sotto-voci). La *gerarchia* di cui parliamo è quella **statica** (tutti i figli dichiarati, indipendentemente da quelli mostrati). Due modi per far scomparire/riapparire un figlio:

- **show / hide**: cambiare la classe CSS aggiungendo/togliendo `display:none`; il figlio resta nell'HTML del genitore, solo non visibile. Preferito quando il figlio va acceso/spento *di frequente e rapidamente*;
- **mount / unmount**: aggiungerlo/toglierlo dal DOM. Preferito per sotto-componenti *so-stanziosi* mostrati solo occasionalmente, o quando la struttura dei figli è generata dai dati e i dati cambiano.

9.3.2 Single Page Application

Le diverse “schermate” dell'app sono tutte figlie di un macro-componente principale, **App**, che monta *un solo figlio alla volta*: quando l'utente “naviga”, App smonta un figlio e ne monta un altro – *non c'è mai una vera navigazione* (che provocherebbe un reload e il reset dell'applicazione); la pagina non contiene mai veri link ipertestuali. Due modi di realizzarlo:

- **gestione manuale**: menu come insiemi di “pulsanti” che montano/smontano i sotto-componenti; semplice ma macchinosa su app articolate;
- **router**: una libreria che permette di definire *route* (analoghe a quelle server-side) con i componenti da montare per ciascuna; il router *intercetta* la navigazione e configura la UI. Vantaggio: preserva all'utente l'illusione di navigare fra pagine (la barra del browser mostra la route attiva anziché la URL dell'unica vera pagina).

9.4 Il pattern MVVM

Definizione – Model–View–ViewModel

- **Model** (modello dati): i dati che l'app elabora e i vincoli che ne regolano fruizione ed elaborazione;
- **View**: ciò che l'utente vede e con cui interagisce (*il DOM*);
- **View-Model**: il *comportamento dei componenti* – legge i dati e li presenta nella view; riceve gli eventi UI, richiede modifiche ai dati, rilegge i dati aggiornati e aggiorna la presentazione.

Il nucleo del pattern è il **legame (binding)** fra View-Model e View, che nei framework è *parzialmente automatizzato*, in due direzioni:

- **property binding**: la View riflette i dati del View-Model; ogni framework ha la sua strategia per accorgersi di quando qualcosa cambia nel view-model e aggiornare la view;
- **event binding**: dalla View verso il View-Model; basato sul principio di callback ed event listener, associa una funzione a un evento UI.

9.5 I componenti React: il ViewModel del MVVM

Definizione – Componente React

Un componente in React è una **funzione** (detta *funzione di rendering*) che costruisce il sottoalbero del DOM che costituirà la *view* del componente.

- I componenti sono scritti in **JSX/TSX**, che estende JS/TS permettendo di *interpolare* HTML e codice (e di importare direttamente i CSS).
- La funzione genera la view *a partire dai dati rilevanti* per il componente; il componente *dichiara* da quali dati dipende la sua view, e ogni volta che quei dati cambiano React invoca di nuovo la funzione, rigenerando la view e sostituendo l'elemento nel DOM: è la base del **property binding** in React.
- Nel generare la view, la funzione specifica anche le *callback* che rispondono agli eventi del DOM, stabilendo il comportamento; le callback possono modificare i dati del componente (provocandone il re-rendering): è la base dell'**event binding**.
- A ogni componente React associa un **tag** con lo stesso nome, usabile nella view di altri componenti come un tag HTML: così si definisce la *gerarchia*.

Il nome della funzione diventa il nome del componente (esportabile per usarlo altrove); l'output deve essere un oggetto **ReactElement** (da un'espressione JSX/TSX); il tipo dell'oggetto-funzione è **FunctionComponent**.

Attenzione – I componenti devono essere funzioni pure

React richiede che l'output della funzione di rendering dipenda *solo dall'input*, e che la funzione *non alteri* strutture dati fuori dal proprio scope. Questo vale per la funzione di rendering in sé, **non** per le callback usate al suo interno!

9.5.1 Espressioni JSX/TSX

```
const elem = <p>Pagina creata da {utente.nome} {utente.cognome}</p>
```

L’HTML è riportato direttamente nel codice (senza virgolette); fra graffe (**interpolazione**) ricompare JS/TS: il codice interpolato deve risolversi in un’espressione, il cui valore completa l’HTML in quel punto.

- Un’espressione JSX/TSX deve risultare in **esattamente un sottoalbero** del DOM (una sola radice): `<h1>...</h1><p>...</p>` non è valida. Se servono più radici, si racchiudono nel **tag vuoto** `<>...</>`: si ottiene un `ReactElement` valido, e il tag vuoto viene tolto all’inserimento nel DOM.
- Si usa il normale HTML con accorgimenti: *tutti i tag vanno chiusi* (anche quelli senza contenuto: ``); gli attributi in kebab-case diventano *camelCase* (`data-value` → `dataValue`: diventano proprietà di oggetti JS, e i nomi di proprietà non ammettono il trattino); `class` diventa **className** (`class` è keyword riservata in JS/TS).

Dove può comparire l’interpolazione:

1. come intero contenuto di un tag: `<ul>{espressione}</ul>`;
2. come porzione di testo: `<p>{nome} ha vinto {x+y} euro!</p>`;
3. come valore di un attributo: `<div className={val}>`.

Nei primi due casi il valore può anche essere un `ReactElement` o un *array di ReactElement*, che vengono inseriti in quel punto del DOM:

```
const lista = [<li>Uno</li>,<li>Due</li>,<li>Tre</li>];
return <ul>{lista}</ul>;
```

Il terzo caso è quello che permette di associare comportamento agli eventi UI (ciò che senza React si faceva con i listener): per ogni evento è definito un attributo corrispondente (`onClick`, `onMouseDown`, `onKeyPress`...) a cui si assegna il listener per interpolazione:

```
<button onClick={(e) => myFunction(e, ...)}>PREMI!</button>
```

9.6 Le props: l’input di un componente

Definizione – Props

Un componente può dichiarare dei parametri che il genitore, nell’istanziarlo, dovrà valorizzare: collettivamente sono le **props**. Il componente prende come argomento un oggetto `props`; la sua “forma” va dichiarata in una *interface* che ne costituisce il tipo. Il genitore assegna i valori al momento dell’istanziatura: i nomi delle props diventano *attributi* del tag del componente.

```
interface BookProps { bIS: BookInBookshelf; }
function Book({bIS}: BookProps): ReactElement {...}

function Bookshelf(...): ReactElement {
  const bookElems = books.map(book => <Book bIS={book} />);
  return (<div className="libreria">{bookElems}</div>);
}
```

Vantaggi evidenti: **riuso** (stesso componente su dati diversi), **configurabilità** (il genitore determina aspetti della view del figlio), **gestione del flusso di informazioni** (ogni componente conosce solo gli aspetti del data model che influenzano la sua view; il genitore “spartisce” i dati fra i figli). Ma l’effetto centrale in React è:

Da ricordare

Se il valore di una prop cambia, React rigenera la view del componente, invocando nuovamente la funzione con i nuovi valori.

Parentesi: object destructuring. Costrutto sintattico frequente nei componenti: `const {x, y} = pippo` dichiara due variabili con i valori delle proprietà omonime di `pippo`. Nei parametri formali: `function fun({x, y}) {...}` – chi invoca passa un unico oggetto con proprietà `x` e `y`, disponibili come variabili locali. Si applica anche agli array.

9.7 Lo state: lo stato di un componente

Con le sole props la view è “statica”: i componenti cambiano al variare delle props, ma non c'è modo di modificarle a partire dagli *eventi UI*. Manca uno **stato** del componente che cambi nel tempo e influenzi la view. Come conciliarlo con componenti che sono funzioni *pure*? (Se un listener modificasse una variabile locale – possibile per via della closure – sarebbe un'operazione fine a sé stessa: al re-render la variabile verrebbe ridichiata e resettata.)

- i componenti *dichiarano* di aver bisogno di variabili di stato (tipo e valore iniziale);
- il valore corrente è conservato in un deposito **esterno ai componenti**, gestito “da React”;
- a ogni esecuzione, il componente ottiene da React il valore corrente, da usare nel determinare la view come gli altri input;
- ottiene anche una **funzione per modificare lo stato**, che *non può essere invocata nel codice del componente* ma solo nelle callback asincrone: la modifica dello stato provoca il re-rendering (nuova esecuzione della funzione), e modificare lo stato dentro il componente causerebbe un **loop infinito**.

Da ricordare

Come per le props: **se il valore di uno state cambia, React rigenera la view**, invocando di nuovo la funzione, che opera sul valore aggiornato.

9.7.1 Lo hook useState

```
const [myState, setMyState] = useState<T>(valoreIniziale);
```

- `useState<T>` prende il *valore iniziale* (di tipo `T`), usato solo la prima volta (quando la variabile di stato non esiste ancora);
- restituisce una coppia (tecnicamente un array di due elementi – la riga sopra è un *array destructuring*): il **valore corrente** e la **funzione setter**;
- il setter riceve un valore di tipo `T` che diventa il prossimo valore dello stato; può anche ricevere una *funzione di aggiornamento* `(T) => T`, che determina il nuovo valore in base al precedente.

Esempio

```
type ContentNavItem = "negozio" | "libreria" | "lend" | "borrow" | "amici";
export function ContentNav(): ReactElement {
  const [activeItem, setActiveItem] = useState<ContentNavItem>("libreria");
  return (<nav className="content-nav">
    <div className={activeItem === "negozio" ? ... }>
      onClick={() => setActiveItem("negozio")}>
```

```

    Negozio
    </div> ... </nav>);
  }

```

9.7.2 I React Hooks

`useState` è uno **hook**: le funzioni della libreria React per “agganciarsi” al processo con cui React gestisce i componenti e influirvi parzialmente (il setter di `useState`, oltre a modificare lo stato, *forza il refresh*). Altri hook: `useReducer`, `useContext`, `useRef`, `useMemo`, `useCallback`, `useEffect`... Riconoscibili dal nome che inizia per `use`; si possono definire hook *custom* a partire da quelli esistenti.

Attenzione

Gli hook **non** si possono usare ovunque: soltanto nello *scope principale* di una funzione-componente o di un custom hook.

9.8 Delega di un evento al genitore

Talvolta lo stato di un componente deve aggiornarsi in seguito a eventi *nei suoi figli*. Esempio: una lista con selezione singola – ogni voce è un componente che si accorge del proprio click, ma per *deselezionare le altre* serve la visione d’insieme, posseduta solo dal componente-lista: sarà lui ad avere la variabile di stato che registra la voce selezionata, e ogni figlio dovrà *avvisarlo*.

Definizione – Delega al genitore

Al verificarsi di un evento nel figlio, viene chiamata una **callback che appartiene al genitore**. Il figlio dichiara di ricevere *fra le sue props* una funzione definita dal genitore: la callback da chiamare al verificarsi dell’evento. Per convenzione, le props di gestione eventi hanno nome `on` + nome dell’evento. La delega può essere totale o parziale (si può chiamare anche una callback del figlio, o una callback del figlio che invoca quella del genitore): il figlio sceglie se ha cambiamenti di stato propri da attuare.

Livelli di astrazione. Quando un componente delega un evento si dice che lo *espone* al genitore; il nome dell’evento esposto va usato nel nome della prop (evento “action” → prop `onAction`). Il figlio può esporre *direttamente* l’evento UI (la voce chiama la callback a ogni click: prop `onClick` – e in effetti anche gli “elementi HTML” in JSX sono componenti interni di React, e i loro attributi sono props!), oppure *elaborare* gli eventi UI e determinare un evento a un **livello di astrazione più alto**: il componente-lista non inoltra tutti i click (con posizione del mouse e tasti premuti), ma chiama la callback solo quando *cambia effettivamente la selezione* (tre click sulla stessa voce = una sola selezione). Espone così un evento più alto, es. “selection” → `onSelection`: sono gli **eventi custom**, specifici dei componenti o dell’applicazione.

9.9 Form in React: binding bidirezionale

In una SPA *non* vogliamo il meccanismo automatico del browser per i form (provoca un reload): li gestiamo con gli strumenti di React. È “form-like” qualunque componente contenga elementi di input (INPUT, SELECT, TEXTAREA); anche i pulsanti di conferma/annullamento possono essere qualsiasi elemento cliccabile. Passi:

1. per ciascun elemento di input si dichiara uno **state** che ne contiene il valore: `<input type="text">` → string; `<select>` → il **value** della OPTION selezionata (string); `<input type="checkbox">` → boolean;
2. si assegna all'input l'attributo **value**, facendolo corrispondere allo state: `<input value={stateVar}>`;
3. si riceve l'evento **onChange** e lo si usa per modificare lo state:

```
<input type="text" value={stateVar}
      onChange={(e) => setStateVar(e.target.value)}>
```

È insieme un property binding (stateVar passata per interpolazione) e un event binding (stateVar settata in conseguenza di un evento): si parla di **doppio binding** o **binding bidirezionale**.

9.9.1 Tre tipologie di componenti form-like

- **Pannello di controllo:** sempre presente nella view; le impostazioni dei controlli si riflettono sulle view degli altri componenti. La delega al genitore avviene *quasi a ogni cambiamento* di stato interno (che si riflette in uno stato del genitore, e quindi nelle props degli altri figli). Es.: filtri per categoria.
- **Modulo:** presente nella view; l'“azione” si compie solo alla pressione del pulsante di conferma – solo allora avviene la delega; gli altri eventi (incluso un eventuale reset) modificano solo lo stato interno. Es.: iscrizione a una newsletter in una colonna laterale.
- **Modal dialog:** *non* presente nella view; appare per chiedere dati e resta finché l'utente non conferma o annulla. La visibilità è *responsabilità del genitore* (un componente invisibile non può cambiare il proprio stato); la delega avviene sia alla conferma sia all'annullamento, così il genitore può far scomparire la dialog. Es.: dialog per i dati di pagamento.

9.10 Il ciclo di rendering di React

1. Inizializzazione. Rendering di `App()`, quindi dei componenti: per ciascuno React determina gli *state*, le *dipendenze* fra state+props del componente e le props dei figli, ed effettua il rendering effettivo (incluso quello dei figli).

2. Cambiamenti di stato. All'invocazione di `setVar` per una variabile di stato `var`: React determina *a quale componente* appartiene lo stato; re-rendering del componente; React determina le conseguenze del cambiamento su altre variabili/espressioni della funzione di rendering e *quali props dei figli risultano modificate*; re-rendering effettivo **solo di ciò che è cambiato**: elementi montati/smontati, elementi con attributi cambiati, figli con props cambiate.

Cosa provoca i cambiamenti di stato? Le *callback UI* (interazione con la UI) e le *funzioni async* (es. una fetch)... ovvero gli **effect**.

9.11 useEffect: gli effetti collaterali del rendering

Definizione – Effetto

Un **effetto** è un blocco di codice che deve essere eseguito in conseguenza di certi cambiamenti di props o state, ma che *non può* essere eseguito direttamente nella funzione-componente – perché influisce sul mondo esterno (violerebbe la purezza), perché può causare un re-rendering (es. cambia lo stato) o comunque interferire col rendering. Viene eseguito come *funzione asincrona* in seguito all'invocazione della funzione di rendering. Esempi: **fetch di un dato**

necessario alla visualizzazione; sottoscrizione di un servizio esterno con callback; logging su DB locale; effetti di animazione.

```
useEffect(funzioneEffetto, dipendenzeEffetto?)
```

- **funzioneEffetto**: la funzione che attua l'effetto;
- **dipendenzeEffetto**: array *opzionale* delle variabili dello scope locale da cui l'effetto dipende (il cui cambiamento ne richiede la ri-esecuzione). **Array vuoto** $\Rightarrow$ eseguito solo al primo rendering; **array non specificato** $\Rightarrow$ eseguito *a ogni* rendering.

La funzioneEffetto viene inserita nella *coda degli eventi asincroni*: eseguita quando il rendering è ormai terminato.

9.11.1 useEffect + useState

Essendo asincrona, la funzioneEffetto può invocare un setter di stato – attenzione però a non innescare un **loop infinito** fra cambi di stato e dipendenze:

- caso semplice: il setState tocca una variabile che *non* influisce sulle dipendenze dell'effetto $\Rightarrow$ nessuna ri-esecuzione;
- caso gestibile: la modifica influisce sulle dipendenze e l'effetto viene rieseguito – allora la modifica non deve avvenire sistematicamente, ma sotto una condizione che prima o poi diventa falsa (un loop, ma finito, come un while).

Il caso più comune (spesso inevitabile): al primo rendering (e talvolta al cambio di una prop) il componente deve fare una **fetch**; la fetch sta in un effetto F; all'arrivo dei dati (then) li comunica con **setState(dato)**; segue il re-rendering. Bisogna specificare le dipendenze di F in modo che il cambiamento di **dato** *non ne provochi la riesecuzione*: fra le dipendenze non ci deve essere **dato**, né alcuna variabile locale calcolata a partire da esso.

9.11.2 La funzione di cleanup

Alcuni effetti devono “fare le pulizie” prima di essere rieseguiti (es. terminare una sottoscrizione o una connessione prima di aprirne un'altra, per evitare notifiche doppie o memory leak). Inoltre un effetto può essere rieseguito per un re-rendering *prima* che l'esecuzione precedente si sia compiuta: es. fetch dipendente da una prop, prop aggiornata, nuova fetch avviata prima che la precedente sia completata – due fetch in corso per due versioni del dato, ordine di arrivo imprevedibile: *il dato obsoleto potrebbe arrivare dopo e sovrascrivere quello corretto*. Infine, un componente può essere smontato prima che un suo effetto si compia: l'effetto cercherebbe di modificare uno state che non esiste più (React rimuove gli state dei componenti non più validi).

Da ricordare – Cleanup

La risposta è la **funzione di cleanup**: la funzioneEffetto deve *restituire* un'altra funzione (senza argomenti né ritorno) che attua annullamento, invalidamento o terminazione dell'effetto. React la invoca: **subito prima di reinvocare il medesimo effetto e subito prima di smontare il componente**. È così importante che, in fase di sviluppo, React invoca tutti gli effetti *due volte di fila*: eventuali mancate pulizie emergono più facilmente (anche se non sempre).

10. Server HTTP e applicazioni server-side

10.1 Il Server HTTP

Definizione – Server HTTP

Un **server HTTP** è un'applicazione eseguita su una macchina detta *host*, che sfrutta (fra le altre cose) i servizi TCP/IP del sistema operativo per ricevere richieste HTTP e rispondervi. Dall'esterno ci si rivolge all'host con l'indirizzo IP (o lo hostname corrispondente); per contattare lo specifico servizio serve la **porta** su cui è in esecuzione. Il suo compito, in breve: restare in ascolto sulla porta assegnata e, all'arrivo di una richiesta, elaborarla e fornire una risposta.

La maggior parte dei server HTTP è in esecuzione sulla porta di default **80** (**443** per HTTPS), che quindi non va esplicitata. In fase di *sviluppo e test* le cose cambiano: i servizi comunicano sullo stesso computer – IP `127.0.0.1`, hostname `localhost` (lo strato TCP/IP sa che destinatario = mittente) – e l'unico modo per differenziarli è la porta: ecco perché spesso si hanno più server HTTP su porte diverse.

10.2 Gestione delle richieste: due modelli

Un server HTTP deve gestire una grande quantità di richieste “in parallelo”: le richieste giungono in *coda* e vengono considerate in ordine di arrivo, ma il tempo di elaborazione T è potenzialmente superiore all'intervallo fra due arrivi (la coda è raramente vuota). Non si può gestire la coda in modo *completamente* sequenziale – sarebbe come un laboratorio di analisi che non accetta un paziente finché tutti quelli che lo precedono non hanno completato gli esami e sono usciti: spreco di risorse per chi eroga e di tempo per chi riceve.

Da ricordare – Modello 1: single-thread asynchronous event loop

Le richieste sono prese in carico in modo sequenziale da un **loop a thread singolo**; qualsiasi elaborazione necessaria deve quindi avvenire **in modo asincrono**:

- le elaborazioni di *I/O* (file system, altri servizi come un DBMS) sono naturalmente asincrone: le esegue il sistema operativo, non il processo del server;
- le elaborazioni *CPU-intensive* vanno gestite generando un thread separato.

Analogia: al banco dell'accettazione c'è *una sola persona*, che esamina la richiesta e la dirige: se è un'attesa (stampa di un referto) avvia l'operazione e chiede di attendere; se richiede attività lunghe chiama un collega che prende in carico l'utente. In entrambi i casi passa subito alla richiesta successiva. Esempi: **Nginx**, qualsiasi server basato su **Node.js**.

Da ricordare – Modello 2: multi-thread

A ogni richiesta il server genera un **thread separato e dedicato**, che la prende in carico e può gestirla anche in modo sincrono. Analogia: ogni volta che un utente entra, appare un nuovo sportello con una persona interamente dedicata a lui. Esempi: **Apache Web**, **Apache Tomcat**.

A un esame superficiale il multi-thread sembrerebbe più costoso ma più efficiente verso gli utenti; in realtà negli ultimi due decenni sono emersi server single-thread persino più efficienti (**Nginx è praticamente nato per dimostrarlo**). Un vantaggio del single-thread: **performance più prevedibili sotto alto carico** – spesso è meglio una performance leggermente peggiore ma pre-

vedibile (si pensi all’attesa alla fermata dell’autobus!). Con le ottimizzazioni attuali i due approcci sono comparabili; resta importante *sapere su quale modello si basa* il server su cui si “monta” la propria applicazione.

10.3 HTTP Server vs Web Server

Da ricordare

Il **Web** è un servizio specifico realizzato tramite HTTP: un server è un *Web Server* se realizza davvero il “core business” – fornire risorse statiche e dinamiche che contribuiscono al WWW, in modo sufficientemente general purpose (di solito si dice che un applicativo “*fa da*” Web Server, perché dipende da come è usato; in linea di principio potrebbe esistere un Web Server non basato su HTTP... servirebbe però il browser corrispondente). **HTTP** è invece un elemento *infrastrutturale*: “Server HTTP” si riferisce al protocollo con cui comunica, non agli scopi. Un server HTTP può non essere usato come Web Server (es. esporre i dati di un DB, fare solo da reverse proxy); un’applicazione server-side che espone un’API HTTP non è certo un Web Server, ma *contiene* necessariamente un server HTTP per ricevere le richieste.

10.4 Le funzioni di un Server Web/HTTP

10.4.1 Fornire contenuti statici e dinamici

La funzione più comune (non l’unica): fornire contenuti statici e dinamici via HTTP – inclusa la possibilità per gli utenti di *inviare* dati da pubblicare (la base del “Web 2.0”) e le vere Applicazioni Web. Le risorse scambiate sono principalmente JSON, ma un’app client-side ha sempre come punto d’ingresso una pagina HTML + CSS + JS: il server che la “pubblica” deve fornire queste risorse (statiche o dinamiche); quasi sempre un minimo di gestione di risorse statiche serve.

10.4.2 Virtual hosting

Un Web Server può ospitare **più host**, ognuno col proprio hostname, avendo (nella maggior parte dei casi) un *unico indirizzo IP*: tutti gli hostname virtuali sono mappati dal DNS sullo stesso IP, quindi il server deve **esaminare la richiesta HTTP** per stabilire quale hostname era stato usato. Alternativa: mappare ogni hostname su una *porta* diversa – ma il DNS non mappa la porta e il browser usa quella di default: serve allora il **reverse proxying**.

Definizione – Reverse proxy

Un **reverse proxy** (per HTTP) è un server HTTP che riceve le richieste (es. sulla porta di default) e *in base al loro contenuto le reindirizza altrove* (es. al server principale sulla porta abbinata allo hostname della richiesta). Molto usato anche per siti/app i cui servizi sono erogati da *molteplici* Web Server: stesso hostname, e il proxy smista in base al pathname. Alcuni server (come Nginx) sono usati principalmente per questo.

(Nota: in una sottorete locale si può usare un IP come alias di un altro, dando a ogni host virtuale il proprio IP; è però meno efficiente, e si preferisce mappare più hostname sullo stesso IP.)

10.4.3 Autorizzazione e autenticazione

Molti Web Server permettono o vietano l’accesso a certi pathname/risorse in base all’*IP del mittente* (es. rete aziendale: risorse interne solo ai computer della rete, sito pubblico per tutti; i dipendenti in smart working usano una **VPN**, che permette di “presentarsi” con un IP della rete locale – legalmente, installando un certificato del proprietario della rete).

Attenzione – Non sono la stessa cosa

Autenticazione = *riconoscere l'identità* dell'utente che richiede una risorsa. **Autorizzazione** = *permettere o vietare l'accesso* a una risorsa: può dipendere da molti fattori, e l'identità è solo uno di essi. Spesso usate in sinergia (per proteggere contenuti privati), ma distinte.

10.4.4 Caching

Il server può memorizzare le risposte HTTP associate alle relative richieste, per *evitare di reperire di nuovo la stessa risorsa* a fronte della stessa richiesta – tanto più se la risorsa è dinamica e costosa da calcolare, o va richiesta a un altro server (tipicamente un reverse proxy fa anche caching). Obiettivi: velocizzare la risposta e ridurre il carico. La risposta non può però essere “la stessa in eterno”: uso, validità e scadenza della cache si **negozano fra client e server tramite gli header HTTP**.

10.4.5 Bandwidth throttling

Quando il server eroga dati che consumano molta banda in uscita, può *ridurre artificialmente* l'ampiezza di banda di una singola risposta, per suddividere la banda fra più stream paralleli (analogia dei rubinetti su un tubo comune). Es.: video streaming in alta qualità – senza throttling, il primo stream occupa tutta la banda e il secondo utente va “a singhiozzo”.

10.4.6 Rewrite engine

Non sempre la URL richiesta dal client corrisponde al pathname interno con cui il server reperisce la risorsa – spesso è voluto: URL interne lunghe e complesse, ma verso i client URL “pulite” ed evocative. Alcuni server includono un **motore di riscrittura**: regole sintattiche che traducono le URL dalla versione “clean” alla “raw” (es. con Apache Web le regole si scrivono nel file `.htaccess`), eventualmente diverse da un sito all'altro.

Da ricordare – Rewrite engine e routing client-side

Con un *router lato client* (es. in React) il framework sostituisce la navigazione del browser: al click su un link, se la URL è una “location” interna, configura la UI senza vera navigazione (che resetterebbe l'app). Ma se l'utente *scrive la URL nella barra*, il browser fa partire la richiesta al server: l'azione corretta è redirigerla sulla pagina dell'applicazione – e per questo serve il **Rewrite Engine**. In fase di sviluppo il problema non si pone: il server HTTP che serve l'app client è interno all'ambiente di sviluppo, dedicato solo ad essa, e sa cosa fare.

10.4.7 Logging

Registrare informazioni sugli scambi HTTP, per scopi *statistici* e di *security*.

10.5 Server HTTP e applicazione server-side: chi contiene chi?

Un'applicazione server-side è un processo che resta in esecuzione, in attesa di processare richieste HTTP ed elaborare risposte. Qual è la relazione con il server HTTP? Due casi emblematici (architetture più complesse sono riconducibili a questi):

Da ricordare – (a) Il Server HTTP “contiene” una o più app

Il server esiste per sé, è installato e spesso fa altro oltre a dare accesso alle app (spesso è un Web Server, e fra le risorse fornite c'è l'app client-side). Il server contiene un *motore di*

esecuzione (es. una JVM) che tiene “accese” le applicazioni: tipicamente crea un ambiente privato per ogni app (il **Web Container**) con una propria istanza del motore, e un *thread separato per ciascuna app* (modello **multi-threaded**). Ogni app dichiara i pathname/route a cui risponde; il server li registra in una **tabella di routing** e, all’arrivo di una richiesta sulla porta di default, la inoltra all’app destinataria invocandone le funzioni, che restituiscono la risposta da inviare al client.

Da ricordare – (b) L’app “contiene” il server HTTP

Il punto d’ingresso è il motore di esecuzione del linguaggio, dentro il quale l’applicazione *si mette in ascolto* su una porta: le librerie del linguaggio offrono un server HTTP di base che l’app istanzia e attiva. Il server interno “avverte” (meccanismo event-driven) all’arrivo di una richiesta e la inoltra a una funzione dell’app, che restituisce la risposta. La differenza chiave: in (a) *il server ha il controllo e lancia l’app*; in (b) *l’app ha il controllo e lancia il server*. In questo modello l’applicazione è un **Application Server basato su HTTP** (può anche fare da Web Server, ma le relative funzionalità vanno incluse esplicitamente, spesso via librerie). Si abbina più facilmente a un server *single-thread*; nulla vieta più app su porte diverse dietro un **reverse proxy** sulla porta di default (avvicinando (b) ad (a)).

10.6 Caratteristiche generali di un’app server-side

Il mondo esterno. Qualunque sia il modello, l’app comunica con l’esterno attraverso il suo ambiente di esecuzione (che offre le librerie per farlo): il *sistema operativo* (in particolare il file system, spesso per logging); nel caso (a), *informazioni di contesto* fornite dal server; *servizi o risorse di rete* – fra cui la connessione a un DBMS.

Elementi e fasi di esecuzione:

- **Middleware:** prima che la richiesta arrivi all’app vera e propria, una o più funzioni effettuano *elaborazioni preliminari*: la tipica funzione di middleware prende la richiesta, opzionalmente la trasforma e la “passa oltre” (alla prossima funzione o all’app) *oppure la blocca* – per questo sono dette anche **filtri**. Usi: respingere richieste non autorizzate, interpretare e trasformare body/parametri, aggiungere informazioni. Utile per operazioni da eseguire *sempre*, per ogni richiesta.
- **Individuazione della richiesta:** analizzare *method + route* per determinare la funzione invocata, ed estrarre l’input (segmento di route parametrica, query string o body).
- **Elaborazione della risposta:** il “core” – l’app esegue la funzionalità e produce una risposta (risultati, o anche solo un acknowledgement).
- **Output della risposta:** la risposta viene inviata (tramite il server HTTP) *in streaming*: la comunicazione verso il client è gestita come buffer, per inviare anche file grandi senza bloccare (anche lato client i dati arrivano in streaming: l’arrivo della risposta segna solo l’inizio del flusso).
- **Errori:** richieste con operazioni non autorizzate, inconsistenze o parametri inaccettabili; oppure eccezioni nell’elaborazione. Va inviata una risposta con status code appropriato (**4xx** o **5xx** a seconda dei casi).

10.7 Il ruolo dei framework server-side

Il ruolo principale dei framework nelle app server-side è **automatizzare parte dei compiti** e rendere più uniformi le interfacce verso il “mondo esterno”: il framework si frappone fra il core dell’applicazione e l’ambiente di esecuzione, facendosi carico di parte dei compiti (quali, come, e come

inserire codice personalizzato dipende dal framework). La scelta del server HTTP è fortemente legata al *linguaggio*, che determina sostanzialmente il framework.

Combo linguaggio/framework più usate nel web (2023): PHP/Laravel, Python/Django, Java/Spring, Python/Flask, Node.JS/Express.

Esempio – La strana storia di PHP

Per decenni PHP è stato il principale linguaggio di back-end – ma per un modello architetturale diverso (pre-AJAX/JSON): il back-end *generava dinamicamente pagine HTML* e PHP semplifica molto questo compito. Esempi tipici: Wordpress e Moodle, i **CMS** (content management system). Queste applicazioni hanno diffusione enorme e vengono ancora aggiornate: PHP resta *di gran lunga il linguaggio più usato nel Web* – il che non significa che sia il più richiesto o quello con cui si scriverebbe oggi un'app ex novo con front-end molto interattivo (AJAX/JSON). Per applicativi aziendali su larga scala la combinazione più richiesta oggi è **Java + Spring / Spring Boot**. Ha senso studiare PHP oggi? Sì, anche se non come primo linguaggio di web development.

11. Jakarta e le Servlet

11.1 Jakarta EE e il Web Profile

Nelle applicazioni web basate sul **Jakarta Web Profile** si parte dal presupposto che sia il *server HTTP* – che esiste come applicativo indipendente – a incapsulare l'applicazione (modello (a) del capitolo precedente). Il server di riferimento è **Apache Tomcat**, basato sul modello *multi-thread*.

Definizione – Jakarta EE e Web Profile

Jakarta Enterprise Edition (Jakarta EE, già *Java EE*) è un insieme di specifiche che estendono Java con librerie, componenti e API per lo sviluppo di applicazioni commerciali, non incluse nel JDK standard. Il **Jakarta Web Profile** è il sottoinsieme orientato allo sviluppo di applicazioni web. Tomcat implementa la specifica del Web Profile: il suo runtime realizza un ambiente di esecuzione per l'applicazione server-side, fatto di classi, interfacce e oggetti Java utilizzabili per realizzare le funzionalità previste.

I due strumenti principali del Jakarta Web Profile:

- le **Servlet**: oggetti i cui metodi ricevono richieste HTTP, elaborano la risorsa richiesta e la inviano in risposta. Lo sviluppatore scrive le proprie classi-Servlet, istanziate da Tomcat all'avvio dell'applicazione; ogni Servlet specifica le route a cui risponde e Tomcat le registra nella propria tabella di routing, inoltrando le richieste alle servlet opportune;
- le **JSP** (Jakarta Server Pages): pagine in un linguaggio che intreccia HTML e frammenti di Java; a ciascuna Jakarta abbina una Servlet che, alla richiesta, usa la JSP per *generare una pagina HTML* da inviare al client – il modello delle app web multi-pagina *orientate al server*. Nel corso non si usano: l'architettura scelta è la single-page con due applicazioni indipendenti (client-side e server-side) che scambiano JSON, dove la UI è costruita dal browser, non generata dal server.

11.2 Architettura di esecuzione di un'app basata su Servlet

Attori principali: il **Server HTTP** (Tomcat), l'**applicazione** (anche più istanze sulla stessa macchina) e il **Web Container**, che fa da interfaccia fra server e applicazioni, implementando la specifica del Web Profile e gestendo il *ciclo di vita* delle servlet.

Il **Server HTTP** riceve le richieste; rispetto alle app installate, valuta tramite il pathname se una richiesta:

- deve andare verso una delle app (tipicamente ogni app è caratterizzata da un segmento iniziale di pathname) → la passa al Web Container;
- va gestita altrimenti (es. servire una risorsa statica);
- riferisce una risorsa inesistente → risponde 404 **Not Found**.

Il **Web Container** è responsabile di:

- *inizializzare* gli oggetti servlet all'avvio del server (o al primo rilascio dell'app);
- *gestire le richieste* per l'applicazione e invocare i servizi della Servlet opportuna (anche se il pathname punta a un'app, la specifica route può non essere gestita: può generarsi comunque un 404);
- *gestire le eccezioni* sorte nell'app, inviando in tal caso 500 **Internal Server Error**;

- *distruggere* le servlet allo spegnimento del server o alla disinstallazione.

11.3 Cos'è una Servlet: la gerarchia di classi

```
package tweb.myfirstapp;
import java.io.*;
import jakarta.servlet.http.*;
import jakarta.servlet.annotation.*;

@WebServlet(name = "helloServlet", value = "/hello-servlet")
public class HelloServlet extends HttpServlet {
    private String message;
    public void init() { /* ... */ }
    public void doGet(HttpServletRequest request,
                       HttpServletResponse response) throws IOException {
        /* ... */
    }
    public void destroy() { /* ... */ }
}
```

Una Servlet è un'istanza di una classe che estende `HttpServlet`. La gerarchia:

- l'**interfaccia Servlet** definisce i metodi che ogni Servlet deve possedere – quelli che servono al Web Container per gestirla:
 - `init(ServletConfig config)`: invocato dal container per inizializzare la Servlet dopo averla creata;
 - `service(ServletRequest req, ServletResponse res)`: invocato quando il container chiede alla Servlet di gestire una richiesta e ottenere la risposta;
 - `destroy()`: invocato prima della chiusura; la Servlet può “fare pulizia” (connessioni aperte, servizi sottoscritti, registrazioni).
- la classe astratta **GenericServlet** implementa **Servlet** dando un'implementazione di base di `init(config)`: memorizza internamente l'oggetto `config` e chiama un metodo `init()` *senza parametri*, che non fa nulla ma può essere ridefinito dalle sottoclassi;
- la classe **HttpServlet** estende **GenericServlet** per il caso HTTP (in teoria esistono Servlet per altri protocolli). Implementa il `service` di base, che: verifica che la richiesta sia HTTP; crea gli oggetti `HttpServletRequest/Response` dagli originali `ServletRequest/Response`; invoca una propria `service` che accetta gli oggetti `Http`; questa esamina il *method HTTP* e chiama il metodo interno **doXXX** corrispondente (`doGet`, `doPost`, ...). I `doXXX` di base non fanno nulla: è compito delle sottoclassi (le Servlet dell'applicazione) implementare quelli relativi alle richieste che vogliono gestire.

11.4 Il ciclo di vita di una Servlet

La vita di una Servlet inizia all'installazione (o, se già installata, all'avvio del server) e si conclude allo shutdown del server o alla disinstallazione; nel mezzo è sempre attiva e pronta a ricevere richieste. Tre momenti: **START**, **SERVICE**, **STOP**.

Da ricordare – START

1 – *Creazione*: il Web Container crea un **ServletContext** (uno, comune a tutta l'applicazione); crea una *singola istanza* per ciascuna Servlet dell'app; per ogni Servlet crea un oggetto **ServletConfig** con i parametri di configurazione (usando fra l'altro le informazioni annotate

con `@WebServlet`) e un riferimento al `ServletContext`; inserisce in un suo *registro* le Servlet e le route a cui rispondono. *2 – Inizializzazione:* per ogni Servlet `sN` con config `cfN`, il container chiama `sN.init(cfN)`, che a sua volta invoca `sN.init()`. Solo al completamento della init la Servlet è considerata attiva e pronta.

Da ricordare – SERVICE

1 – Matching & Routing: il container effettua un **matching** fra la URL ricevuta e gli `urlPattern` dichiarati dalle Servlet, determinando la servlet “vincente”; l’algoritmo ha una strategia per i conflitti (es. se una URL corrisponde a due pattern, “vince” quello *più lungo*). Se non c’è matching: 404 Not Found. *2 – Predisposizione:* vengono predisposti gli oggetti `HttpServletRequest/Response` che estendono i `ServletRequest/Response` di partenza (la request contiene tutte le informazioni sulla richiesta: URL, method, header...). *3 – Invocazione del servizio:* il container chiama `s.service(req, res)` di `HttpServlet` con gli oggetti predisposti; questo invoca a sua volta il `doXXX` appropriato per la richiesta in esame.

Da ricordare – STOP

1 – Cleaning up: per ogni servlet il container chiama `destroy()` (di base non fa nulla; da ridefinire se la init ha fatto cose che vanno “disfatte”, es. chiudere una connessione a un DB). *2 – Garbage collection:* vengono eliminati gli oggetti Servlet, i `ServletConfig` e il `ServletContext`.

12. Spring e Spring Boot

12.1 Il ruolo di Spring e di Spring Boot

Definizione – Spring

Ai fini del corso, il ruolo di **Spring** è evitare al programmatore la scrittura di *boilerplate code* (codice ripetitivo di inizializzazione e gestione delle componenti di base, slegato dalla logica dell'applicazione). Lo sviluppatore realizza dei **request handler**, fornendo al framework informazioni su: *a quali richieste HTTP* ciascun handler risponde (method/route) e *quali dettagli della richiesta* gli servono (parametri della query string, body, segmenti parametrici). Il framework invoca l'handler corretto e gli fornisce le informazioni richieste. Gli handler sono raggruppati in classi dette **controller**; dentro Spring (lontano dagli occhi dello sviluppatore) ciascun controller corrisponde di fatto a una *servlet* di Jakarta. Spring fornisce molto altro: ad esempio facilita enormemente l'accesso ai dati su DB.

Definizione – Spring Boot

La piattaforma Spring è molto versatile e configurabile finemente – ed è proprio questa “laicità” a rendere complessa la configurazione dei progetti. **Spring Boot** estende Spring adottando un approccio “dogmatico” su alcune scelte architetturali: semplifica, velocizza e rende più efficiente lo sviluppo, a patto di essere compatibili con tali scelte. Punti principali:

- **dependency starter**: pacchetti di dipendenze standard che evitano di specificare una per una le librerie (si possono comunque aggiungere dipendenze custom);
- **Tomcat incapsulato**: Spring Boot include e gestisce internamente un'installazione di Tomcat – l'applicazione è di fatto *stand-alone*, con deploy molto semplificato (niente configurazione pre-esistente del server);
- **configurazioni predefinite** per molte funzionalità di Spring, adeguate alla maggior parte dei casi.

12.1.1 App stand-alone e main class

Senza Spring Boot (solo Spring, o solo Jakarta) l'applicazione non avrebbe una “main class”: sarebbe un insieme di classi istanziate dal Web Container o dall'ambiente Spring, e invocate quando serve. Per eseguirla bisognerebbe installare Tomcat, compilare e impacchettare (*bundle*) i `.class`, effettuare il *deploy* in Tomcat – tutto fortemente dipendente da S.O. e configurazione. Spring Boot lo evita: restano le “classi operative” istanziate dall'ambiente Spring, ma c'è anche una **main class** che invoca le librerie di Spring perché eseguano una versione *embedded* di Tomcat con l'applicazione già rilasciata al suo interno – basta un normale run:

```
@SpringBootApplication
public class MyApplication {
    public static void main(String[] args) {
        SpringApplication.run(MyApplication.class, args);
    }
}
```

12.1.2 Inizializzazione di progetto: SpringInitializr e Maven

Lo scheletro si genera con lo **SpringInitializr** (interfaccia web `start.spring.io` che produce uno zip; IntelliJ ha il generator “Spring Boot” integrato); come dipendenza di partenza si specifica *Spring Web*. La gestione di progetto e dipendenze è affidata a **maven** (in alternativa gradle; maven è più lineare e semplice, sufficiente per i nostri scopi): la configurazione e le dipendenze stanno nel file **pom.xml**. Elementi salienti del pom: il `<parent>` (si eredita la configurazione di base da un pom genitore fornito da Spring Boot, `spring-boot-starter-parent`); le `<properties>` (es. `<java.version>17</java.version>`); le `<dependencies>`, che fanno riferimento ai pacchetti *starter* (es. `spring-boot-starter-web`). I **dependency starter** sono “pacchetti di dipendenze” corrispondenti alle necessità più comuni delle applicazioni enterprise, richiedibili in blocco dal pom.xml; sul repository di maven (`mvnrepository.com`) se ne può esaminare il contenuto.

12.1.3 Cenni al linguaggio XML

Definizione – XML

XML (eXtensible Markup Language) è un *meta-linguaggio* di annotazione: non definisce quali tag usare (come HTML) ma permette di *definire e usare i propri*. Un XML è **ben formato** se: i tag sono sequenze alfanumeriche fra parentesi angolari; a ogni `<nome>` corrisponde `</nome>`; un tag può chiudersi solo se tutti quelli aperti al suo interno sono chiusi (struttura ad albero); c'è un unico tag radice. XML struttura le informazioni esprimendone il significato: le rende parsificabili automaticamente e più leggibili per gli umani (si provi a togliere i tag dal pom.xml!).

Per uno scopo specifico si può definire l'insieme dei tag ammissibili e le loro relazioni: un **XML-Schema**. Un XML può riferirsi a uno schema nell'intestazione e risultare *valido* o meno rispetto ad esso (ben formato $\neq$ valido). Il pom.xml riferisce lo schema `maven-4.0.0`. *Nota*: HTML sarebbe quasi definibile con un XML-Schema; l'unico ostacolo sono i tag senza chiusura (`<img>`). L'HTML ottenuto imponendo la chiusura di tutti i tag è detto **XHTML**: ha convissuto con HTML, poi è stato soppiantato da *HTML Living Standard*, dove i tag senza chiusura sono validi – ne consegue che un HTML *non* è un XML ben formato.

12.2 Le annotazioni Java

Definizione – Annotazioni

Le **annotazioni** sono il sistema con cui il programmatore fornisce *metadati* su un elemento che sta dichiarando. Sintassi: `@Nome` (UpperCamelCase), eventualmente con coppie chiave-valore fra tonde: `@Ann("someinfo")`, `@Ann(elem1="v1", elem2="v2")` (se c'è un solo valore, associato per convenzione alla chiave `value`, la chiave si può omettere). Si possono annotare classi, campi, metodi, parametri. Java ne definisce alcune predefinite; nuove annotazioni possono essere definite dagli sviluppatori (nel nostro caso arrivano dai package inclusi via pom.xml).

Da ricordare

Le annotazioni sono **puramente dichiarative**: non *fanno* qualcosa, forniscono informazioni. I destinatari possibili:

- *altri sviluppatori* (segnalazioni, es. `@Deprecated`);
- *il compilatore*, per verifiche (es. `@Override`: se il metodo non è davvero un override, errore);
- *servizi runtime*, per attuare comportamenti sugli elementi annotati – ed è questo a ren-

derle centrali in Jakarta e ancor più in Spring: il framework, grazie alla **Java Reflection API**, legge le annotazioni su classi, metodi, parametri e campi, e attraverso di esse capisce come, dove e quando utilizzarli, come invocarne le funzionalità e come dotarli di comportamenti aggiuntivi. L'uso delle annotazioni in Spring si avvicina a una *estensione del linguaggio*.

12.3 Il pattern MVC in Spring

Spring struttura le applicazioni col pattern **Model–View–Controller** e definisce tre tipologie di componenti: **@Controller**, **@Service**, **@Repository**.

- Il **controller** è il *punto di ingresso*: riceve la richiesta HTTP e la elabora, facendo da *mediatore* fra una o più *view* e gli aspetti *model* di sua pertinenza. È responsabile dell'**API**, che possiamo vedere come la descrizione della *view* che il controller mette a disposizione.
- La **view** qui ha un'accezione ampia: non (solo) una presentazione grafica, ma un *punto di vista sui dati* – un insieme di oggetti JSON è a tutti gli effetti una vista – inclusi i modi messi a disposizione per richiederne aggiornamenti. Ogni controller realizza una vista e la descrive tramite la sua API: quali JSON restituisce a quali richieste, quali aggiornamenti offre, quali informazioni servono per richiederli.
- Il **model** contiene la *business logic* e l'accesso ai dati (inclusa la persistenza): conosce i dati, la loro strutturazione e i vincoli di coerenza, sa organizzarli e aggiornarli. I dati in quanto tali sono nei componenti **@Repository** (legati alla persistenza); i controller possono accedervi così come sono (organizzazione DB-like) oppure tramite i **@Service** di business logic, che aggregano e strutturano i dati, ne garantiscono la consistenza e offrono operazioni di più alto livello.

12.4 Inversion of Control e Dependency Injection

Definizione – IoC e DI

Con il **controllo diretto**, se al componente X servono i servizi di Y è responsabilità dello sviluppatore creare un'istanza di Y e passarla a X. Con l'**inversion of control** (controllo rovesciato) esiste un *framework* che mantiene tutti i “servizi” e li fornisce a chi dichiara di necessitarne; il framework può anche esporre *versioni astratte* dei servizi, fornendo implementazioni diverse a seconda del contesto, cambiandole senza scossoni per chi le usa, o fornendo versioni *mock* per il testing. L'IoC è spesso realizzato tramite la **dependency injection**: il componente che vuole usare certi servizi *lo dichiara*, ad esempio specificandoli come parametri del proprio *costruttore*; poiché è il framework a costruire il componente, al momento della creazione gli “inietta” le istanze appropriate (già create in precedenza). Risultato: minore accoppiamento fra utilizzatore e fornitore di un servizio.

12.5 L'architettura di Spring: IoC container e bean

Definizione – Bean e IoC container

Spring è un ambiente di esecuzione *modulare* al cui centro sta l'**IoC container**, che crea e gestisce oggetti chiamati **bean**: oggetti in esecuzione che assolvono alle funzionalità richieste da un modulo Spring. Due caratteristiche distintive:

- i bean sono tutti **singleton**: esiste sempre un solo bean di una data classe (per questo

si indica un bean con la sua classe); crearne due risulterebbe in un’eccezione;

- l’IoC container sa **iniettare** un bean (passarlo come parametro) agli altri bean che ne fanno richiesta.

Esempi: un `@RestController` è un bean usato da Spring per rispondere alle richieste su certe route; un `@Service` è un bean usato non da Spring ma dai controller o da altri service; altri bean predefiniti gestiscono aspetti come l’invio di un `500 Internal Server Error` se il nostro codice genera un errore.

Tutti i bean vengono creati dall’IoC container *all’avvio dell’applicazione*, rispettando le dipendenze dichiarate (via dependency injection); **circoli viziosi di dipendenze** (A dipende da B che dipende da C che dipende da A) generano un’eccezione. Due modi principali di dichiarare un bean:

- annotare una classe con **@Component** o una delle annotazioni *derivate* (`@RestController`, `@Service`, `@Repository`, `@Configuration`...): l’IoC container ne crea una singola istanza all’avvio; i parametri del costruttore stabiliscono le dipendenze da altri bean, creati prima e poi passati al costruttore (DI *a livello di costruttore*). Vale anche per classi che estendono classi-bean già definite nelle librerie;
- annotare una classe con **@Configuration** e uno o più metodi con **@Bean**: il container interpreta “il metodo crea/restituisce un bean”, lo esegue all’inizializzazione e inserisce l’oggetto fra i bean disponibili; le dipendenze si dichiarano fra i *parametri del metodo* (DI *a livello di metodo*).

Il ruolo dei moduli. Un modulo Spring (Spring Web, JPA, Spring Security...) è un insieme di package che estende Spring:

- fornendo *bean infrastrutturali* con comportamenti di base – es. Spring Web offre il bean **DispatcherServlet** che ascolta le richieste HTTP e le redirige ai `@RestController` e agli handler appropriati (o restituisce 404), e questo bean esiste indipendentemente dal fatto che definiamo dei controller;
- fornendo *bean di configurazione* che influenzano quelli infrastrutturali – es. i bean responsabili dell’interpretazione dei body delle richieste e della serializzazione dei body delle risposte;
- fornendo *insiemi di annotazioni* per estendere il comportamento di base con i propri bean e farli comunicare con gli altri (`@RestController`, `@GetMapping`, `@PathVariable`...).

Programmare in Spring significa quindi: scrivere i propri bean che interagiscono con quelli esistenti tramite le annotazioni dei moduli importati; modificare (estendendoli, ridefinendoli o configurandoli) i bean di configurazione per ottenere il comportamento desiderato.

12.6 Il @RestController: rispondere alle richieste HTTP

Definizione – Controller e request handler

Un **controller** è una classe annotata `@RestController`; un **request handler** è un metodo di un controller annotato con `@RequestMapping` o con uno degli *shortcut* `@GetMapping`, `@PostMapping`, ...

`@RequestMapping` e gli shortcut ammettono proprietà per *filtrare* le richieste:

- **value**: un pathname (String) o un insieme di pathname (array);
- **method**: il method della richiesta (`RequestMethod.GET`, `RequestMethod.POST`, ...); gli shortcut non lo ammettono perché è già implicito (`@GetMapping` $\equiv$ `@RequestMapping(method=RequestMethod.GET)`);

- **params**: query param che la richiesta deve avere, eventualmente vincolati nel valore ("param=value");
- **headers**: header richiesti, anche qui vincolabili ("header=value").

`@RequestMapping` si può applicare anche *all'intera classe* controller: quel pathname diventa il *prefisso* dei pathname dei singoli handler.

12.6.1 `ResponseEntity<T>`: inviare risposte JSON

La classe **`ResponseEntity`** permette agli handler di restituire oggetti attorno a cui Spring confeziona una risposta HTTP, di default di tipo `application/json`. Metodi statici di convenienza:

- `ResponseEntity.ok(data)`: status 200, body = data;
- `ResponseEntity.notFound().build()`: status 404. Serve `build()` perché `notFound` restituisce una risposta *parziale*, ancora configurabile (a differenza di `ok`, che è già compiuta);
- `ResponseEntity.status(200).headers(httpHeaders).body(data)`: come `ok`, ma configurando gli header con un oggetto `HttpHeaders` (coppie proprietà/valore).

Attenzione

Spring determina quali campi di un oggetto serializzare/deserializzare a partire dai **getter pubblici** della classe (indipendentemente dalla presenza dei setter).

12.6.2 Ricevere le proprietà della richiesta

Un request handler specifica *come parametri del metodo* le proprietà della richiesta che vuole conoscere, comunicando a Spring la corrispondenza tramite annotazioni:

- **`@RequestParam`**: un parametro della *query string*; se il nome coincide col parametro formale non serve altro; `@RequestParam(name="parName")` per nomi diversi; `@RequestParam(required=false)` se può essere assente;
- **`@PathVariable`**: un *segmento parametrico* del path, indicato nel pathname del mapping fra graffe { } con un nome che deve coincidere col parametro formale;
- **`@RequestBody`**: il *body* della richiesta; il parametro formale deve essere del tipo corretto per la deserializzazione della JSON string;
- **`@RequestHeader(HttpHeaders.HEADER)`**: il valore dell'header specificato (`HttpHeaders` definisce costanti coi nomi degli header HTTP).

12.7 I `@Service`: business logic

Un servizio di business logic è una classe annotata **`@Service`**. Spring ne crea *una e una sola istanza*; la classe può essere fatta come si ritiene opportuno (i principi sono quelli della buona OOP più che di Spring): l'idea è offrire un insieme *coeso e coerente* di funzionalità che compone, con altri service e con i repository, la business logic. Per usare un service dentro un altro componente (controller, altro service, repository) si usa la **dependency injection**, nella forma più comune: prevedere il service come *parametro del costruttore* del ricevente, il cui codice lo memorizza in un campo. I costruttori di service/controller/repository possono prendere parametri *purché iniettabili* (altri bean), e senza dipendenze circolari.

12.7.1 Suddivisione delle responsabilità

Da ricordare – Chi fa cosa

- I **@RestController** gestiscono le richieste HTTP: scelgono, in base a parametri (**@PathVariable**, **@RequestParam**), **@RequestBody** e altre caratteristiche, quali funzionalità di business logic attivare; decidono, in base alla risposta ottenuta, che tipo di risposta HTTP inviare e quali dati ne costituiranno il body. In sostanza: *garantiscono che l'API sia quella concordata con l'app client-side*.
- I **@Service** attuano la business logic – le vere funzionalità dell'app. Servono i controller ma anche *l'un l'altro*; frequentemente operano I/O sullo strato di persistenza usando i **@Repository**, talvolta “cucendo insieme” più richieste a più repository per una funzionalità complessa.
- I **@Repository** garantiscono la corretta *persistenza* dei dati loro affidati: ognuno gestisce una entità ben precisa; non sanno nulla della business logic né dell'API – conoscono solo la tabella a cui fanno riferimento.

12.8 Entity e Data Transfer Object

La rappresentazione delle entità nella *persistenza* (le **@Entity** e i **@Repository** di JPA) è usata dalla business logic di back-end, ma **non è (quasi mai) la stessa usata nella comunicazione col front-end** – e non si dovrebbe ipotizzare che lo sia. Motivi:

- a volte back-end e front-end hanno proprio una *visione diversa* del modello;
- a volte il back-end *non vuole esporre* il modello così com'è (si pensi alla password dell'oggetto User!);
- le **@Entity** sono spesso collegate fra loro (**@OneToMany**, **@ManyToMany**...): oggetti molto “ricchi”, con collegamenti che si collegano ad altri collegamenti; verso il front-end si vuole spesso una versione più *shallow*, riservandosi di mandare altri dati su richiesta;
- viceversa, a volte si vogliono *aggregare* più **@Entity** in un unico oggetto con i soli aspetti che interessano al front-end.

Definizione – DTO

Le classi che rappresentano la “visione” sulle entità da trasferire al front-end sono i **Data Transfer Object**; è in generale opportuno definire i propri. La responsabilità della conversione Entity ↔ DTO è del *Controller*; la prassi più comune è definire una classe **DTOMapper** con i metodi di conversione, a cui il controller si appoggia.

13. Persistenza dei dati: JPA

13.1 Interfacciarsi a un DB relazionale

Come si eseguono query SQL da un'applicazione? Un DB Server espone i suoi servizi tramite: una *CLI* (command line interface), eventuali *GUI client*, e soprattutto una **API** che rende i servizi accessibili ad applicazioni esterne (locali o remote via TCP). L'API è *specifica del DB* e viene resa disponibile tramite **driver**: librerie che rendono le operazioni invocabili in un dato linguaggio (driver C++, Python, JavaScript, Java...).

Definizione – JDBC

JDBC è un insieme di classi e interfacce a cui i driver Java *devono conformarsi*, rendendo il codice Java **indipendente dal DB**: basterà installare il driver giusto (per Postgres, MySQL, MariaDB...) senza cambiare il codice.

13.2 ORM, JPA, Hibernate

Definizione – ORM

L'**ORM** (Object-Relational Mapping) è una tecnica per mappare i costrutti dei DB relazionali in costrutti object-oriented. Uno strumento di ORM fornisce un'API che permette di accedere alla persistenza *con un modello a oggetti* anziché relazionale: si legge e scrive creando/eliminando oggetti e invocando metodi, anziché con query SQL.

Definizione – JPA e Hibernate

JPA (Jakarta Persistence API) è parte di Jakarta EE: una *specifica* che descrive come gestire dati relazionali da Java. JPA **non implementa** un ORM: è la specifica dell'interfaccia che un ORM per Java deve rispettare – così lo sviluppatore usa l'API di JPA senza essere vincolato a uno specifico *provider*. **Hibernate** è il provider JPA preferito da Spring Boot (esiste anche come ORM indipendente); la cosa più semplice è richiedere via Maven il set `spring-boot-starter-data-jpa`, che include proprio Hibernate. Hibernate interagisce col DB tramite **JDBC**: fra le dipendenze va incluso anche il *driver JDBC* del DBMS scelto (nel corso: Postgres).

Lo “stack” completo: *Spring App* → *JPA* → *Hibernate ORM* → *JDBC* → *DB*.

13.3 Le @Entity: descrivere entità e relazioni

Definizione – Entity

Un'**entità** del dominio in JPA corrisponde a una *classe*, tipicamente mappata su una *tabella* del DB; le istanze sono mappate su uno o più record; i campi esprimono proprietà rappresentabili come colonne (o come tabelle relazionali, per i campi che esprimono relazioni). La classe deve avere un **costruttore di default**: può averne altri per il nostro codice, ma quando JPA crea le istanze caricandole dal DB usa quello di default e poi imposta i campi. Si descrivono entity e relationship con le annotazioni; *della traduzione in tabelle si occupa JPA* – ma conoscere la rappresentazione sottostante aiuta a sfruttarla al meglio.

13.3.1 Le annotazioni principali

- **@Entity**: la classe è un'entità del DB gestita da JPA;
- **@Table(name = "table_name")**: dettagli della tabella (di solito il nome);
- **@Id**: il campo è la *chiave primaria*;
- **@GeneratedValue(strategy = GenerationType.STRATEGY)**: la chiave è generata dal DBMS, con la strategia indicata; per semplicità si usa **GenerationType.AUTO**, che demanda la scelta al DBMS;
- **@Column**: dettagli della colonna – **name** (se diverso dal campo), **nullable** (true/false), **unique** (true/false).

13.3.2 Le relazioni

- **Uno-a-Molti** fra A (l'uno) e B (i molti): si annota un campo in B con **@ManyToOne** (di tipo A) e un campo simmetrico in A con **@OneToMany** (una **Collection** di B), specificando **mappedBy = "owner"** dove *owner* è il nome del campo di B annotato **@ManyToOne**;
- **Molti-a-Molti** fra A e B: campi simmetrici in entrambe le entity, entrambi **@ManyToMany**, entrambi **Collection** (*meglio Set che List*); uno dei due è l'*owner* e aggiunge **@JoinColumn** che specifica la *join table* da usare; l'altro indica con **mappedBy** il nome del campo che “possiede” la relazione.

13.4 I @Repository e le operazioni CRUD

Definizione – Repository

Per operare su una tabella si definisce un **@Repository**: un'interface che estende **JpaRepository<E, Id>** (generico nella classe entity E e nel tipo Id della chiave primaria). **JpaRepository** definisce già i metodi CRUD di base, quindi spesso l'estensione può restare *vuota*:

- **E save(E entity)**: salva l'entity e *restituisce l'entity risultante* (che può essere diversa da quella di partenza!). Se l'id è già nel DB è una *update*, altrimenti una *create*;
- **Iterable<E> saveAll(Iterable<E>)**: salva e restituisce un elenco;
- **Iterable<E> findAll()**: tutte le entity della tabella;
- **Optional<E> findById(Id id)**: l'entity con quell'id, se presente;
- **void deleteById(Id id) / void delete(E entity)**: eliminano, se presente.

13.4.1 Le query derivate

Aggiungendo al repository dichiarazioni di metodi che seguono una precisa *naming convention*, JPA **deriva automaticamente la query SQL** che li implementa:

```
findByAuthorAndTitleContainsOrderByTitleAsc(String author, String title)
deleteTop1ByAuthorBeforeIgnoreCase(String author)
countDistinctByTitleStartsWith(String title)
```

Struttura del nome:

- **operazione**: cosa si vuole fare (sostanzialmente **find**, **delete**, **count**);
- **modificatore dell'operazione**: limita o condiziona i risultati (**Top1**, **Distinct...**);
- **By**: separa il “soggetto” della query dal predicato;
- **specificatori di predicato**: una colonna + un'operazione logica (se omessa, uguaglianza) per filtrare in base al valore; *per ogni predicato serve un parametro in ingresso* col valore da verificare;

- **operatori logici:** And / Or combinano più specificatori;
- **modificatori del predicato:** IgnoreCase (sullo specificatore che lo precede), AllIgnoreCase (sull'intero predicato), OrderBy + colonne + Asc/Desc (ordinamento dei risultati).

13.5 Il ciclo di vita delle @Entity

Da ricordare – Gli stati: new, managed, detached, removed

1. **New:** l'entità è appena creata in memoria (con `new`): JPA non la gestisce e *il DB non ha idea che esista*; le modifiche ai campi non hanno effetto sul DB.
2. **Managed:** quando l'entità viene salvata con `repo.save(e)` (create) diventa *gestita*; lo stesso accade *estraendola dal DB* con un metodo del repository (es. `findById`). Finché è managed, **tutte le modifiche ai campi dell'oggetto si riflettono automaticamente nel DB** (il cosiddetto *dirty checking*). Resta managed *finché non termina la transazione corrente*.
3. **Detached:** al termine della transazione l'entità è “staccata”: le modifiche *non* si riflettono più sul DB. Per riagganciarla si chiama di nuovo **save**: poiché l'id è già nel DB, la save farà una *update* (anziché una create).
4. **Removed:** cancellando con `repo.delete(e)` l'entità è “rimossa”: non appena la transazione termina, scompare dal DB.

Attenzione – Quando termina una transazione?

Se non altrimenti specificato, la transazione è **limitata ai singoli metodi del repository**: chiamando `repo.save(e)`, l'entità è managed *durante* l'esecuzione di save, ma appena save restituisce *non lo è più* (subito detached). Lo stesso per `findById` e gli altri metodi di ricerca.

13.5.1 @Transactional

Definizione – @Transactional

Un metodo **@Transactional** garantisce che le entità restino managed *fino al termine del metodo*: le modifiche vengono trasferite nel DB (*commit*) al termine se il metodo ha successo, o annullate (*rollback*) se fallisce. È per questo che i metodi di un `@Service` sono *tipicamente* `@Transactional`.

Attenzione

La transazione è effettiva **solo se il metodo è invocato da un altro componente che inietta il servizio via Dependency Injection**: solo così il metodo viene eseguito “sotto il controllo” di Spring, che può farlo precedere e seguire dalle operazioni che garantiscono la transazionalità.

13.5.2 save: persist o merge

save può fare due cose diverse:

- **persist**, se l'entità *non ha id*: aggiorna direttamente l'oggetto fornito in input;
- **merge**, se l'entità *ha già un id*: recupera dal DB una versione managed, la aggiorna coi valori della versione detached, e *restituisce la nuova entità recuperata*.

Attenzione

In alcuni casi `save` restituisce lo *stesso* oggetto ricevuto, in altri **no**: buona norma lavorare sempre con l'entità *restituita* da `save`.

14. CORS, sessioni, filtri e autenticazione

14.1 Cross-Origin Resource Sharing (CORS)

Definizione – Origine e CORS

L'**origine** di un'applicazione client-side è il server HTTP che l'ha fornita. Si parla di **cross-origin resource sharing** quando un'app client-side vuole effettuare una fetch da una app server-side che risiede su un'*origine diversa*.

14.1.1 La preflight request

Quando si verifica una richiesta cross-origin, il browser – *prima* di far partire la fetch vera e propria – effettua una richiesta detta **preflight**, con method HTTP **OPTIONS**: un test preliminare per capire se la richiesta effettiva potrà essere recepita dal server. Nella preflight il browser descrive la richiesta che vorrebbe fare con tre header:

- **Access-Control-Request-Method**: quale sarà il method;
- **Origin**: l'origine dell'app client-side;
- **Access-Control-Request-Headers**: gli header della richiesta che verrà effettuata.

Il server risponde alla preflight dichiarando i propri vincoli con gli header:

- **Access-Control-Allow-Methods**: i method permessi;
- **Access-Control-Allow-Headers**: gli header permessi;
- **Access-Control-Allow-Origin**: le origini permesse;
- **Access-Control-Allow-Credentials**: se è permesso l'invio di credenziali (essenzialmente i cookie di sessione).

Il browser così sa se la richiesta verrà accettata e quali header dovrà inserire. **Se la preflight viene respinta, è il browser stesso a bloccare la richiesta originale**; se la mandasse avanti, sarebbe l'app server-side a rispondere con un errore (es. 405 Method Not Allowed o 403 Forbidden a seconda della violazione).

14.1.2 @CrossOrigin

Per default i server HTTP **bloccano** le richieste cross-origin (politica *noCORS*). È un problema in fase di sviluppo, quando abbiamo *due* server HTTP (uno serve l'app client-side, l'altro è embedded nell'app server-side): anche sullo stesso host (localhost) girano su *porte diverse* ⇒ origini diverse.

Approccio semplificato (da usare *solo in sviluppo su localhost*): annotare con **@CrossOrigin** i controller (o i singoli handler) per cui permettere le richieste cross-origin – in sviluppo, tipicamente tutti. Effetto: quando la richiesta è diretta a uno dei nostri controller viene permessa *indipendentemente* da origine, method e header – una sorta di “salvacondotto universale”. Solo l'invio di **credenziali** resta bloccato. (Le richieste che non corrispondono a nessun handler restano soggette a noCORS: vengono bloccate in preflight prima ancora di stabilire se sono valide.)

14.2 Le sessioni

Definizione – Sessione

Quando c'è un'interazione prolungata fra due “parti” (utente e applicazione, client e server, applicazioni peer-to-peer) diventa rilevante il concetto di **sessione**: l'idea che la “conversazione” abbia *uno stato e uno storico*, così che a ogni passo si possa fare riferimento a quanto già detto e fatto, senza ripetere o ricapitolare.

Da ricordare – Perché serve uno stratagemma

Il problema nasce da due ragioni:

1. il protocollo HTTP è di per sé **stateless**: non prevede alcuna idea di stato dello scambio – ogni transazione (richiesta + risposta) è indipendente dalle altre; l'unico modo di “simulare” uno stato sarebbe riportare a ogni passo tutte le informazioni rilevanti già scambiate;
2. un server HTTP gestisce **molte conversazioni con client diversi**: se il client non si rende riconoscibile, il server non può sapere “a che punto” della conversazione si trova, perché è in punti diversi a seconda dell'interlocutore.

Serve quindi “costruire” *sopra* HTTP un meccanismo che permetta a client e server di sapere che stanno avendo una conversazione prolungata e di mantenerne lo stato.

14.2.1 Sessioni tramite cookie

Definizione – Cookie

Un **cookie** è un blocco di dati che il server manda al client *nel primo scambio* della loro conversazione; a ogni scambio successivo, il client *aggiunge il cookie alla propria richiesta*, così il server identifica il particolare “thread” di conversazione con quel client.

Il server *potrebbe* memorizzare le informazioni direttamente nel cookie; di solito però nel cookie si mettono solo le informazioni per *riconoscere l'utente e identificare univocamente la conversazione* (un id di sessione), mentre **lo stato vero e proprio resta sul server** (in memoria o nel DB), indicizzato da quell'id: così si evita di appesantire richieste e risposte HTTP.

14.2.2 Gestione della sessione in Spring Boot

Per attivare i cookie di sessione si configura `application.properties`:

```
server.servlet.session.cookie.name=LAB_SESSION_COOKIE
server.servlet.session.cookie.http-only=true
server.servlet.session.cookie.secure=true
```

- il *nome* identifica il cookie (visibile negli strumenti di sviluppo del browser);
- **http-only**: il browser e il codice client-side *non hanno accesso* ai contenuti del cookie;
- **secure**: il cookie viene trasmesso *cifrato*.

Per lavorare con la sessione si *inietta nei request handler* l'oggetto **HttpSession**, tramite cui:

- scrivere: `session.setAttribute(key, value);`
- leggere: `session.getAttribute(key);`
- invalidare (anche prima del timeout): `session.invalidate();`

- impostare il timeout: `session.setMaxInactiveInterval(secs)` – il timeout indica per quanto tempo il cookie può restare *inutilizzato* (nessuna nuova richiesta con quel cookie); scaduto, il cookie non è più valido e dovrà essere generato uno nuovo.

14.2.3 La collaborazione del client (e il CORS con credenziali)

Perché il meccanismo funzioni, l'app client-side deve *rimandare il cookie a ogni richiesta*. È il comportamento di default del browser **solo per richieste same-origin**: in situazione *cross-origin*, per default le credenziali *non* vengono inviate. Per ottenerne l'invio, nelle opzioni della fetch si specifica:

```
credentials: "include"      // default: "same-origin"
```

Lato server, però, `@CrossOrigin` così com'è non basta più:

1. la risposta alla preflight deve includere `Access-Control-Allow-Credentials: true` ⇒ `@CrossOrigin(allowCredentials = "true")`;
2. per ragioni di sicurezza, **se le credenziali sono ammesse il protocollo HTTP impedisce di abilitare tutte le origini in modo acritico**: l'impostazione di default (`Allow-Origin: qualsiasi`) non è più valida – vanno *elencate esplicitamente* le origini ammesse (una **whitelist**):

```
@CrossOrigin(origins = "http://localhost:5173", allowCredentials = "true")
```

14.3 I filtri

Definizione – Filtro

Un **filtro** in Spring Boot/Jakarta è una funzione attraverso cui passano le richieste HTTP *prima* di giungere ai request handler: viene chiamata per *ogni* richiesta, e può decidere se *inoltrarla* nella **filter chain** (la catena di filtri, che culmina nell'invocazione dell'handler) oppure *respingerla* con uno status code di errore. Uso tipico: verificare che la richiesta soddisfi dei requisiti.

Un filtro è una classe che **implementa l'interfaccia `Filter`** ed è annotata **@Component**: così Spring Boot lo include nella filter chain (che comprende anche filtri già presenti nel framework). L'ordine di esecuzione si controlla con **@Order(N)**.

```
@Component
@Order(1)
public class AuthFilter implements Filter {
    @Override
    public void doFilter(ServletRequest servletRequest,
                        ServletResponse servletResponse,
                        FilterChain filterChain)
                        throws IOException, ServletException {
        HttpServletRequest request = (HttpServletRequest) servletRequest;
        HttpServletResponse res = (HttpServletResponse) servletResponse;
        ...
        filterChain.doFilter(servletRequest, servletResponse); // inoltra
        ...
        res.sendError(HttpServletResponse.SC_FORBIDDEN);        // blocco
    }
}
```

I parametri di `doFilter`:

- **ServletRequest**: la richiesta nella sua forma “grezza”, prima che Spring la elabori; rappresenta una generica richiesta di rete – per le informazioni specifiche HTTP serve il *type casting* a `HttpServletRequest`;
- **ServletResponse**: la risposta “grezza”, in cui Spring inserirà le informazioni degli handler; anche qui casting a `HttpServletResponse`;
- **FilterChain**: la catena a cui inoltrare i due oggetti, per passare il testimone al prossimo filtro o, se i filtri sono finiti, all’handler.

Per **inoltrare**: `filterChain.doFilter(request, response)`. Per **bloccare**: non si invoca la chain e si usa la response per inviare un errore: `response.sendError(HttpServletResponse.SC_XXX)`.

14.4 Basic Authentication con filtri e sessioni

Con filtri e sessioni si può “cucinare” un meccanismo di **autenticazione di base**. Versione semplificata (e per nulla sicura): username e password viaggiano *in chiaro come query parameters*; una versione un po’ più sicura li mette nel *body* di una POST e usa **HTTPS**.

Schema:

- si distingue un utente autenticato da uno non autenticato tramite un **attributo salvato in sessione** (es. lo username);
- per una generica richiesta, **il filtro verifica se la sessione contiene l’attributo**: se sì, la fa passare; altrimenti risponde **401 UNAUTHORIZED**;
- **eccezione: la richiesta di login**, che deve passare in ogni caso. Arriva al request handler, che (tramite un opportuno `@Service`) verifica le credenziali: se corrette, crea in sessione l’attributo che segnala l’autenticazione; altrimenti risponde **401 UNAUTHORIZED**;
- il **logout** si realizza semplicemente *invalidando la sessione*;
- il server deve esporre anche una route, accessibile pure ai non autenticati, che permetta all’app client-side di *verificare lo stato corrente dell’autenticazione*.

15. Server-side app con Node.js ed Express

15.1 Node.js: il server HTTP di base

Node.js incarna il modello (b) del capitolo 10: *l'app contiene il server HTTP*, con gestione **single-thread ad asynchronous event loop**. Le librerie di Node forniscono l'oggetto `http`, con cui creare un server (`createServer`) e attivarlo in ascolto su una porta:

```
import http from "http";
import {IncomingMessage, ServerResponse} from "node:http";
const porta = 8555;
const serverFun = async (req: IncomingMessage, res: ServerResponse) => {
  res.setHeader("Content-Type", "text/html; charset=utf-8");
  res.statusCode = 200;
  res.write(`<h2>${req.method} - ${req.url}</h2>`);
  res.write(`<pre>${JSON.stringify(req.headers)}</pre>`);
  if (req.method === "POST") {
    res.write(`<h3>Body</h3>`);
    const body = await text(req);
    res.write(body);
  }
  res.end();
};
http.createServer(serverFun).listen(porta);
```

Il metodo `createServer` prende come argomento *la funzione che riceve le richieste ed elabora le risposte* – è lei ad attuare le funzionalità dell'applicazione. Deve avere la forma `(req: IncomingMessage, res: ServerResponse) => ...`.

15.1.1 IncomingMessage: l'oggetto request

Permette di conoscere le caratteristiche della richiesta:

- `req.method`: string ("GET", "POST", ...);
- `req.url`: string; contiene la URL *assoluta* (senza hostname/porta);
- `req.headers`: oggetto di coppie chiave-valore con gli header.

Se la richiesta è POST, il **body** si legge usando `req` come *stream di input*. Il modo più semplice: gli **stream consumer** di Node – funzioni che “consumano” uno stream e restituiscono una *Promise* dell'oggetto contenuto; va usato il consumer adatto al Content-Type: `text` legge lo stream come testo semplice, `json` lo legge come JSON string e la deserializza, ecc. (Una trattazione approfondita degli stream di Node esula dal corso, ma va approfondita per usare Node sul serio.)

15.1.2 ServerResponse: l'oggetto response

Permette di impostare la risposta:

- `res.statusCode` / `setStatus(statusCode)`: lo status code (200, 404, ...);
- `res.setHeader(name, val)`: il valore di un header;
- il **body** si scrive usando `res` come *stream di output*: `res.write(text)` per inviare testo, chiudendo poi con `res.end()`;

- alternativa: *inoltrare un intero stream di input* sull'output con `req.pipe(res)` – in questo caso non serve terminare lo stream (termina da solo alla fine dell'inoltro).

15.2 Express

```
import express from "express";      // FRAMEWORK
const app = express();              // APP OBJECT, INCLUDE IL SERVER HTTP
const port = 8883;
// MIDDLEWARE
app.use(express.static("public"));
app.use(express.json());
// REQUEST HANDLERS
app.get("/test/:pr1/:pr2",
  async (req: express.Request, res: express.Response) => { ... })
app.post("/test/:pr1/:pr2",
  async (req: express.Request, res: express.Response) => { ... })
// SERVER ACTIVATION
app.listen(port, () => {
  console.log("Server running on port " + port);
});
```

L'oggetto `app`, creato tramite il framework **express**, *include il server http* e fornisce funzionalità aggiuntive. Configurata l'app, la si attiva con `app.listen(port, host?, callback)`: se non si specifica lo `host`, di default è `localhost`; la `callback` viene eseguita quando il server inizia a ricevere richieste.

15.2.1 Il middleware in Express

In generale il middleware è una funzione

```
(req: express.Request, res: express.Response, next: express.NextFunction) =>
  void
```

Il suo ruolo: elaborazioni preliminari su richiesta o risposta, e decidere se la richiesta può “proseguire l'iter” – in tal caso invoca la funzione `next()` ricevuta come parametro; altrimenti, invece di `next()`, invia una *risposta di errore*.

Nell'esempio, due middleware forniti da `express`:

- `express.static("public")`: i contenuti della cartella indicata vengono *serviti staticamente*;
- `express.json()`: *trasforma la richiesta* pre-analizzando il `body` (se c'è) come JSON string, deserializzandolo e memorizzandolo nella proprietà `body` dell'oggetto `req`.

Si può subordinare un middleware alla struttura della route: `app.use(routeExpression, middleware)` lo applica solo se la URL corrisponde alla `routeExpression` – una stringa che esprime una route (o un insieme di route), con eventuali *segmenti parametrici* preceduti da `:` e il carattere jolly `*`: `/il/mio/pa*th/:nome/:id`.

15.2.2 I request handler in Express

L'applicazione si configura specificando un **request handler** per ogni *method+route* dell'API da esporre. L'oggetto `app` ha un metodo di registrazione per ogni method HTTP: `app.get()` per le route in GET, `app.post()` per quelle in POST, e così via. Il primo parametro è sempre la route (anche qui con segmenti parametrici e jolly: a rigore l'handler è associato a un *insieme* di route); il secondo è l'handler, una funzione `(req: express.Request, res: express.Response) => void`.

express.Request è più ricco del corrispettivo di Node base, perché *pre-elaborato* da express: i parametri sono già estratti –

- **req.query**: oggetto chiave-valore con i parametri della *query string*;
- **req.params**: oggetto chiave-valore con i *segmenti parametrici* della route;
- **req.body**: il body della request, se presente e se interpretato tramite un opportuno middleware (es. `express.json()`).

express.Response prevede anch'esso funzionalità aggiuntive: il metodo **send** invia la risposta in un'unica soluzione, curando anche di *chiudere lo stream* al termine, e *rileva automaticamente* le caratteristiche della risposta (Content-Type, Content-Length) impostando i relativi header. Status code e invio in un'unica riga per concatenazione:

```
res.status(code).send(content)
```